

CEBU INSTITUTE OF TECHNOLOGY - UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description

For

**RigMD: A Symptom-Guided Diagnostic and Decision Support
System
for Non-Technical Windows Desktop PC Users**

Team Code: 2526-sem2-it332-11

Document Version: 1.0

Published Date: 22 May 2026

Prepared by:

Libato, Michael Grant

Macansantos, Axcelson

Ruperez, Raymart

Maestrado, Ralph Keane

Labaya, Godwin

Change History

Version	Date	Description	Author(s)
1.0	22 May 2026	Initial release of Software Design Description for RigMD.	Team 2526-sem2-it332-11

Signature

Name	Signature / Date
Libato, Michael Grant	
Macansantos, Axcelson	
Ruperez, Raymart	
Maestrado, Ralph Keane	
Labaya, Godwin	

Preface

This Software Design Description (SDD) presents the architectural and detailed design of RigMD - A Symptom-Guided Diagnostic and Decision Support System for Non-Technical Windows Desktop PC Users. The document was prepared by Team 2526-sem2-it332-11 of the College of Computer Studies at Cebu Institute of Technology University in partial fulfillment of the requirements for IT332.

RigMD is a downloadable Windows desktop application that assists individual desktop PC owners in identifying probable internal causes of Windows-observable performance symptoms and deciding on an appropriate, clearly classified next action. The system integrates six modules: System Profile, Structured OS Symptom Intake, Internal Diagnostic Interpretation Engine, Advisory Action, Transparent Result Dashboard, and Session History and Recurring Pattern Detection.

This document is intended for use by the development team during implementation, by faculty evaluators during design review, and by future developers who may extend or maintain the system. The detailed design section has been prepared with labeled placeholders for UI/prototype images, component tables, object-oriented diagrams, sequence diagrams, and data design entries.

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions and Acronyms

1.4 References

2. Architectural Design

3. Detailed Design

Module 1 - System Profile and System Overview Module

1.1 Home Dashboard and Quick Actions

1.2 System Profile Detection

Module 2 - Structured OS Symptom Intake Module

2.1 Guided Diagnostic Intake

Module 3 - Internal Diagnostic Interpretation Engine

3.1 Diagnostic Result Summary

3.2 Supporting Diagnostic Details

3.3 Recommended Actions and Decision Confidence

Module 4 - Advisory Action Module

4.1 Warning Signs Reference

Module 5 - Transparent Result Dashboard and Report Output Module

5.1 Reports

Module 6 - Session History and Recurring Pattern Detection Module

6.1 Diagnostic History

6.2 Recurring Patterns

1. Introduction

1.1 Purpose

This Software Design Description defines the architecture and detailed design of RigMD, a symptom-guided diagnostic and decision support desktop application for Windows. The document specifies the system components, module boundaries, transaction flows, user interface preparation, front-end and back-end component placeholders, object-oriented design placeholders, and data design placeholders. It serves as the primary technical reference for development, integration, and future maintenance of the system.

RigMD helps individual non-technical Windows desktop PC owners interpret Windows-observable performance symptoms and arrive at a clear, action-classified decision about their next troubleshooting step. The system guides users from system profile detection to structured symptom intake, diagnostic interpretation, advisory output, report generation, and history review.

1.2 Scope

This document covers the design of the RigMD desktop application as scoped in the approved SRS. The application targets individual Windows desktop PC users, including students, gamers, beginner PC builders, budget desktop owners, content creators, and general home users who experience Windows-observable performance symptoms but lack the technical background to diagnose them independently.

The system is limited to functional or partially functional Windows desktop PCs. Fully non-functional systems are handled only through a boot-status triage path and are routed to a professional inspection recommendation. The system does not perform physical hardware testing, voltage measurement, component substitution, or direct replacement and upgrade recommendations.

The application is designed as a self-contained Python desktop application using CustomTkinter for the graphical user interface, SQLite for local data storage, Windows system information libraries for profile detection, and the Gemini API for constrained plain-language advisory explanation generation.

1.3 Definitions and Acronyms

Term / Acronym	Definition
RigMD	A symptom-guided PC diagnostic and decision support application for non-technical Windows desktop users.
SDD	Software Design Description - this document.

Term / Acronym	Definition
SRS	Software Requirements Specification - the requirements document this SDD is derived from.
GUI	Graphical User Interface - the desktop interface layer built with CustomTkinter.
API	Application Programming Interface.
Gemini API	The AI model API used for constrained plain-language advisory explanations.
SQLite	A lightweight local database used for profile records and diagnostic session history.
S.M.A.R.T.	Self-Monitoring, Analysis, and Reporting Technology - storage health monitoring data used as an observable warning indicator.
WMI	Windows Management Instrumentation - Windows interface used for local system information retrieval.
CustomTkinter	Python GUI framework used to implement the RigMD desktop interface.
DSS	Decision Support System.
BSOD	Blue Screen of Death - Windows stop-error screen indicating critical system failure.

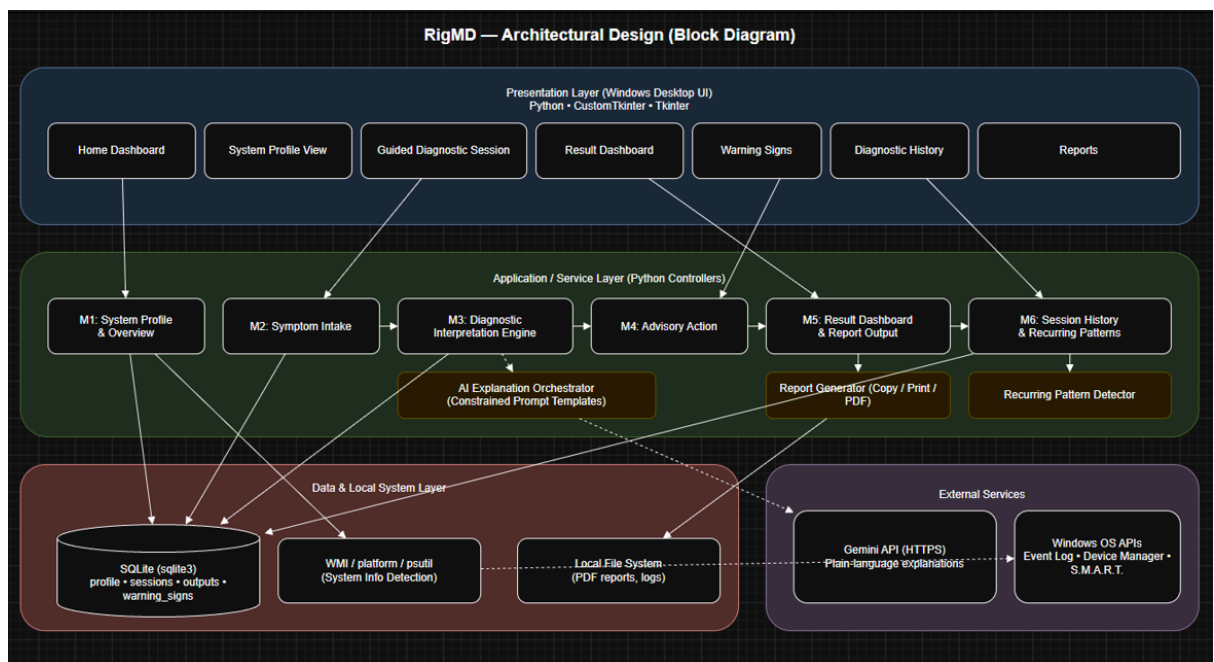
1.4 References

- RigMD Software Requirements Specification, Team 2526-sem2-it332-11, Semester 2, AY 2025-2026.
- RigMD Project Proposal (Weeks 7-8), Team 2526-sem2-it332-11.
- Kostopoulos, G., Davrazos, G., & Kotsiantis, S. (2024). Explainable artificial intelligence-based decision support systems: A recent review. *Electronics*, 13(14), 2842.
- Ravi, M., Negi, A., Bommi, N. S., & Rouf, N. (2025). Evolution of AI-driven decision making with decision support systems, expert systems, recommender systems, and XAI. *IETE Technical Review*, 42(4), 428-465.
- Brooke, J. (1996). SUS: A quick and dirty usability scale. In *Usability Evaluation in Industry* (pp. 189-194). Taylor and Francis.
- Python 3.x Documentation. <https://docs.python.org/3/>
- CustomTkinter Documentation. <https://customtkinter.tomschimansky.com/>

- Google Gemini API Documentation. <https://ai.google.dev/>
- SQLite Documentation. <https://www.sqlite.org/docs.html>

2. Architectural Design

RigMD follows a local-first Windows desktop architecture. The user interacts with a CustomTkinter-based presentation layer. The application layer manages system profile detection, structured symptom intake, diagnostic rule mapping, advisory action classification, report generation, and recurring pattern detection. The local data layer stores system profile records, diagnostic sessions, outputs, reports, and recurring patterns using SQLite. The AI explanation layer is limited to constrained Gemini API calls for plain-language advisory explanation generation.



Technology Stack Summary

Layer	Technology	Design Purpose
Presentation Layer	CustomTkinter	Provides the Windows desktop user interface.
Application Logic Layer	Python modules	Handles symptom intake, diagnostic mapping, advisory classification, report generation, and recurring pattern logic.
System Information Layer	WMI / Python system information libraries	Retrieves detectable system profile values where supported.
Data Persistence Layer	SQLite	Stores profile data, diagnostic sessions, recurring patterns, and report records locally.

Layer	Technology	Design Purpose
AI Explanation Layer	Gemini API	Generates constrained plain-language explanations from structured diagnostic outputs.
Report Output	Local print / copy / PDF export support	Produces technician-ready diagnostic summaries from saved sessions.

3. Detailed Design

Each module below follows the required SDD structure: User Interface Design, Front-end Components, Back-end Components, Object-Oriented Components, Class Diagram, Sequence Diagram, and Data Design. The user interface sections are prepared for UI/prototype insertion, while the component, diagram, and data design tables are intentionally left blank for later completion after the final implementation components are confirmed.

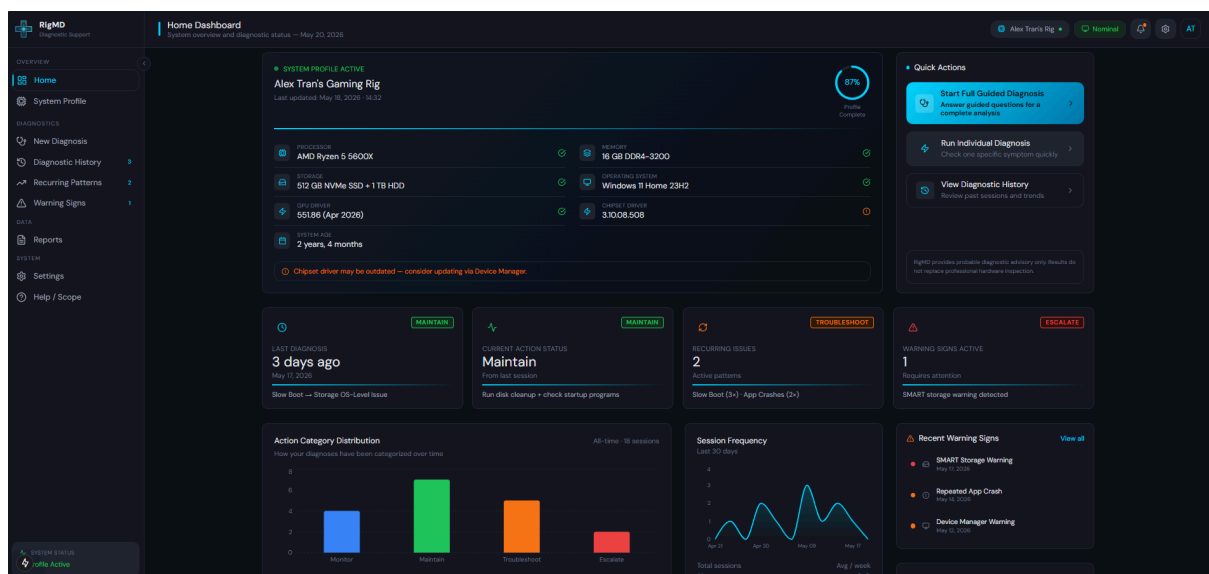
Module 1 - System Profile and System Overview Module

This module provides the first system overview and manages the desktop PC system profile used as diagnostic context. It includes the Home Dashboard and System Profile Detection transactions.

1.1 Home Dashboard and Quick Actions

User Interface Design

The Home Dashboard is the main overview screen shown after the application launches. It summarizes the active system profile, latest diagnostic result, recurring issue count, warning signs, and overall action status. The interface also provides quick actions for starting a guided diagnosis, running an individual diagnostic path, viewing diagnostic history, and checking warning indicators. This screen serves as the user's starting point for understanding the current PC health status.



Front-end Components

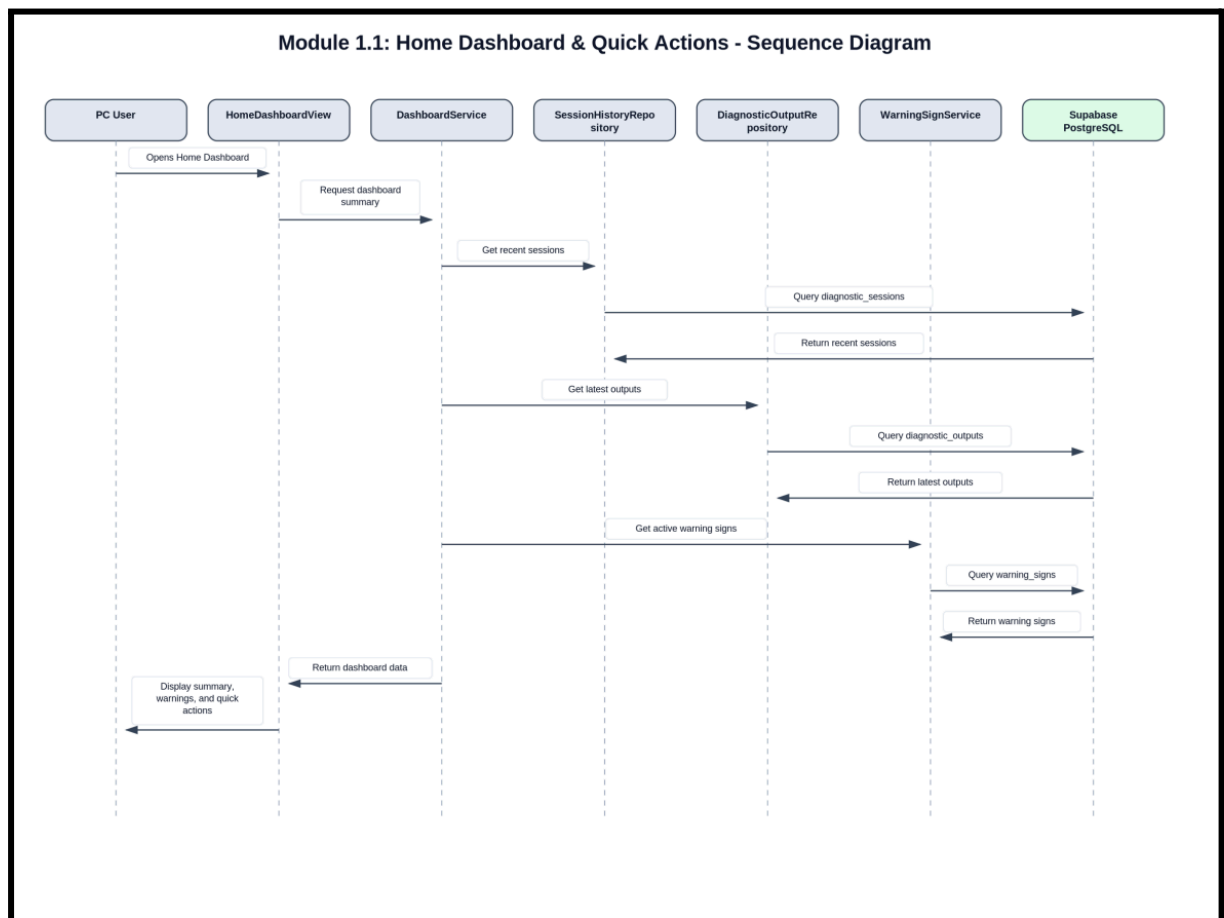
Component Name	Description and Purpose	Component Type / Format
HomeDashboardView	Displays the main dashboard, system status summary, recent activity, and active warning signs.	CustomTkinter Screen / Frame
QuickActionPanel	Provides shortcut buttons for starting a new diagnosis, viewing history, and opening reports.	CustomTkinter Button Group
SystemStatusSummaryCard	Shows the current high-level status of the user's PC based on latest available data.	CustomTkinter Card
RecentActivityCard	Displays the most recent diagnostic sessions and activity trends.	CustomTkinter Card
ActiveWarningSignsCard	Shows warning signs detected or recently connected to diagnostic results.	CustomTkinter Card

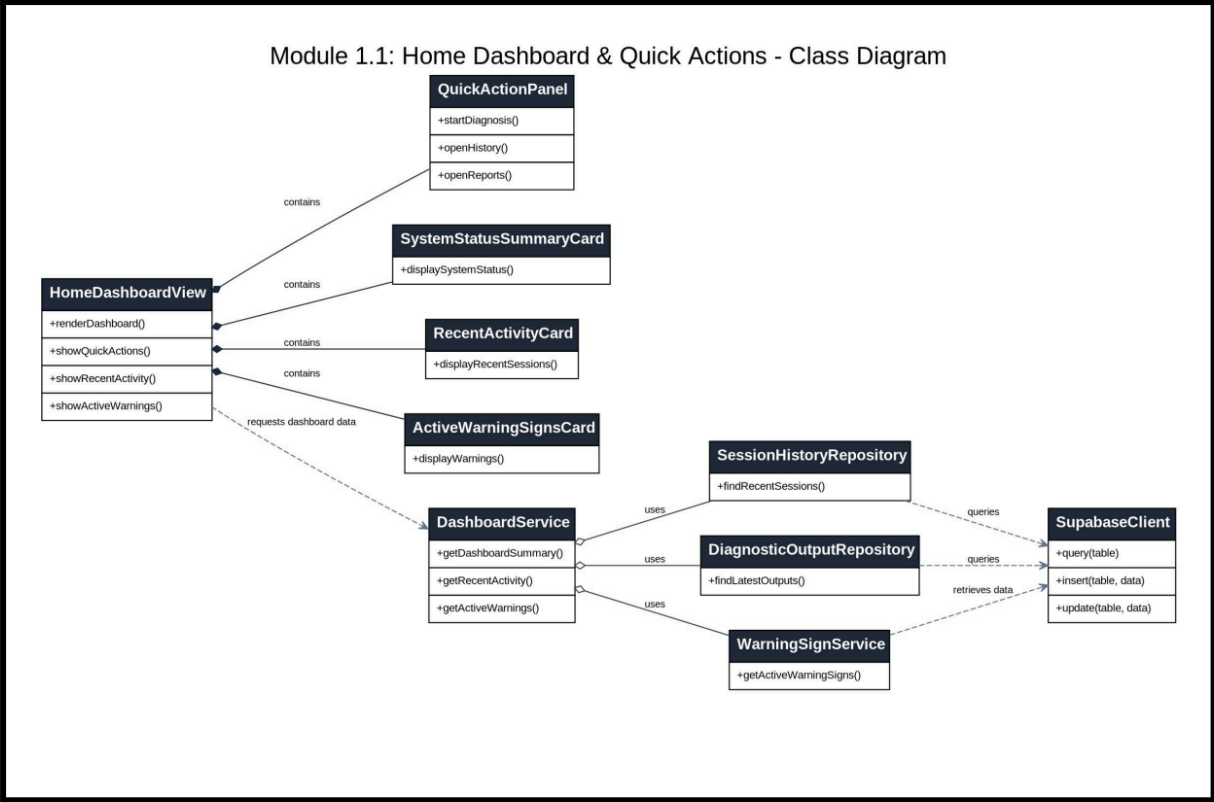
Back-end Components

Component Name	Description and Purpose	Component Type / Format
DashboardService	Collects dashboard data such as latest sessions, warning signs, and summary counts.	Python Service Class

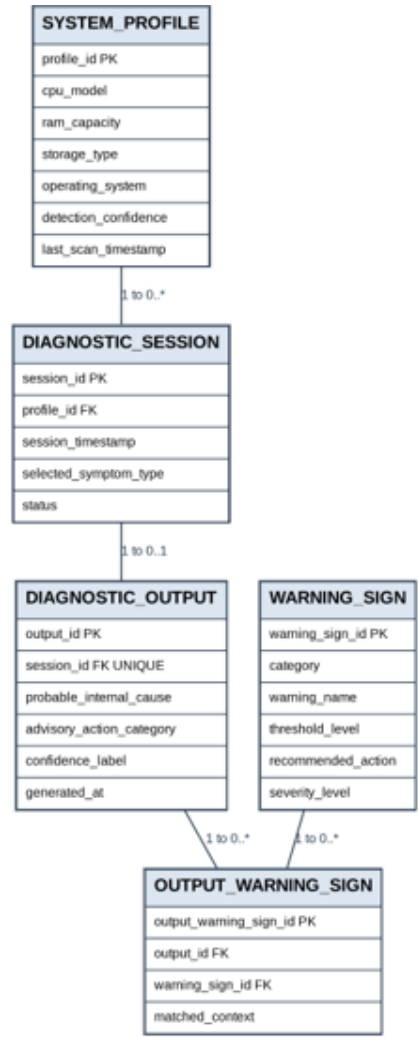
SessionHistoryRepository	Retrieves recent diagnostic sessions from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
DiagnosticOutputRepository	Retrieves latest diagnostic outputs for dashboard display.	Supabase PostgreSQL Repository Class
WarningSignService	Retrieves active or relevant warning signs for display.	Python Service Class

Object-Oriented Components





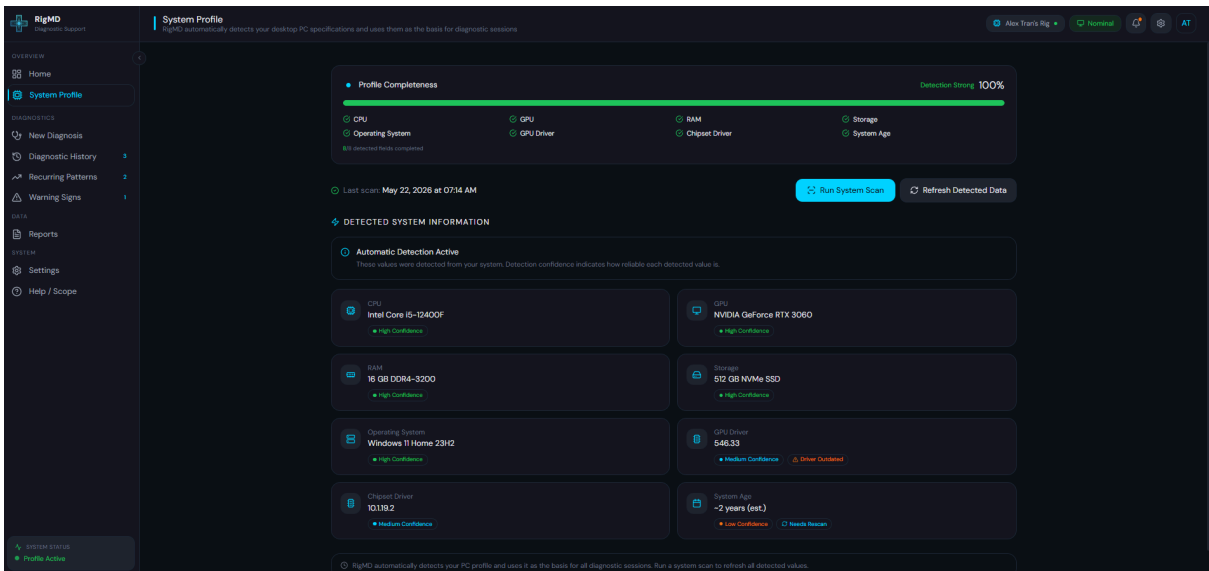
Data Design / ERD or Schema for Home Dashboard and Quick Actions



1.2 System Profile Detection

User Interface Design

The System Profile Detection screen displays the hardware and software information detected from the user’s Windows desktop PC. The interface focuses on automatically detected data such as CPU, GPU, RAM, storage, operating system version, GPU driver version, chipset driver version, system age, scan status, and detection confidence. Manual editing is not included in this screen. If detection is incomplete or low confidence, the interface should clearly mark the affected field and allow the user to refresh the detected data.



Front-end Components for System Profile Detection

Component Name	Description and Purpose	Component Type / Format
SystemProfileView	Displays detected CPU, RAM, storage, OS, GPU driver, chipset driver, and system age.	CustomTkinter Frame / Screen

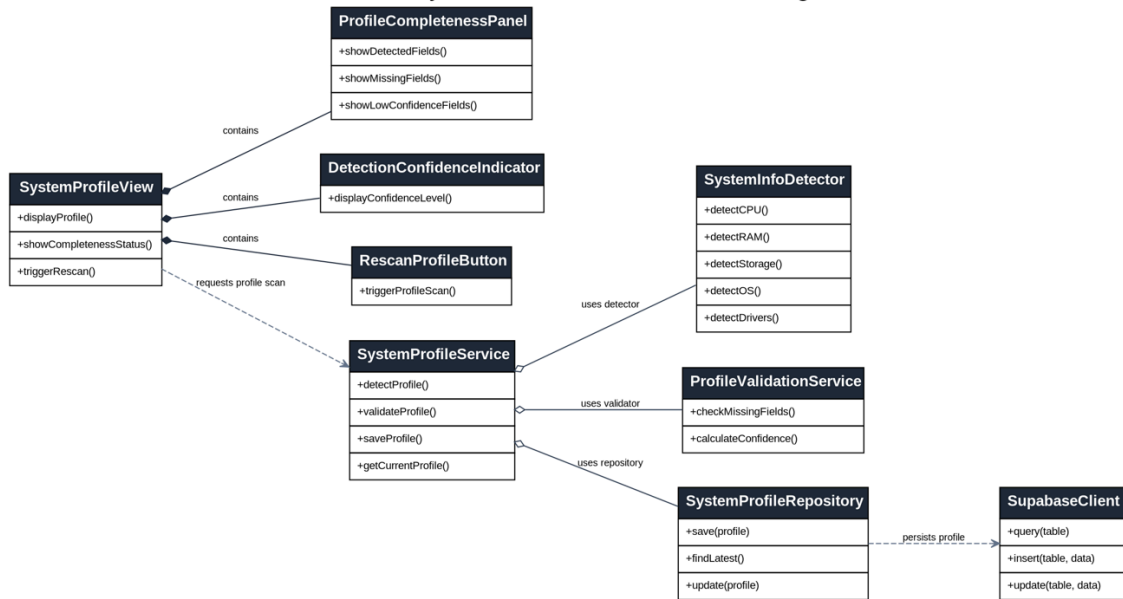
ProfileCompletenessPanel	Shows whether profile fields are detected, missing, or low-confidence.	CustomTkinter Card
RescanProfileButton	Allows the user to scan the system profile again.	CustomTkinter Button

Back-end Components for System Profile Detection

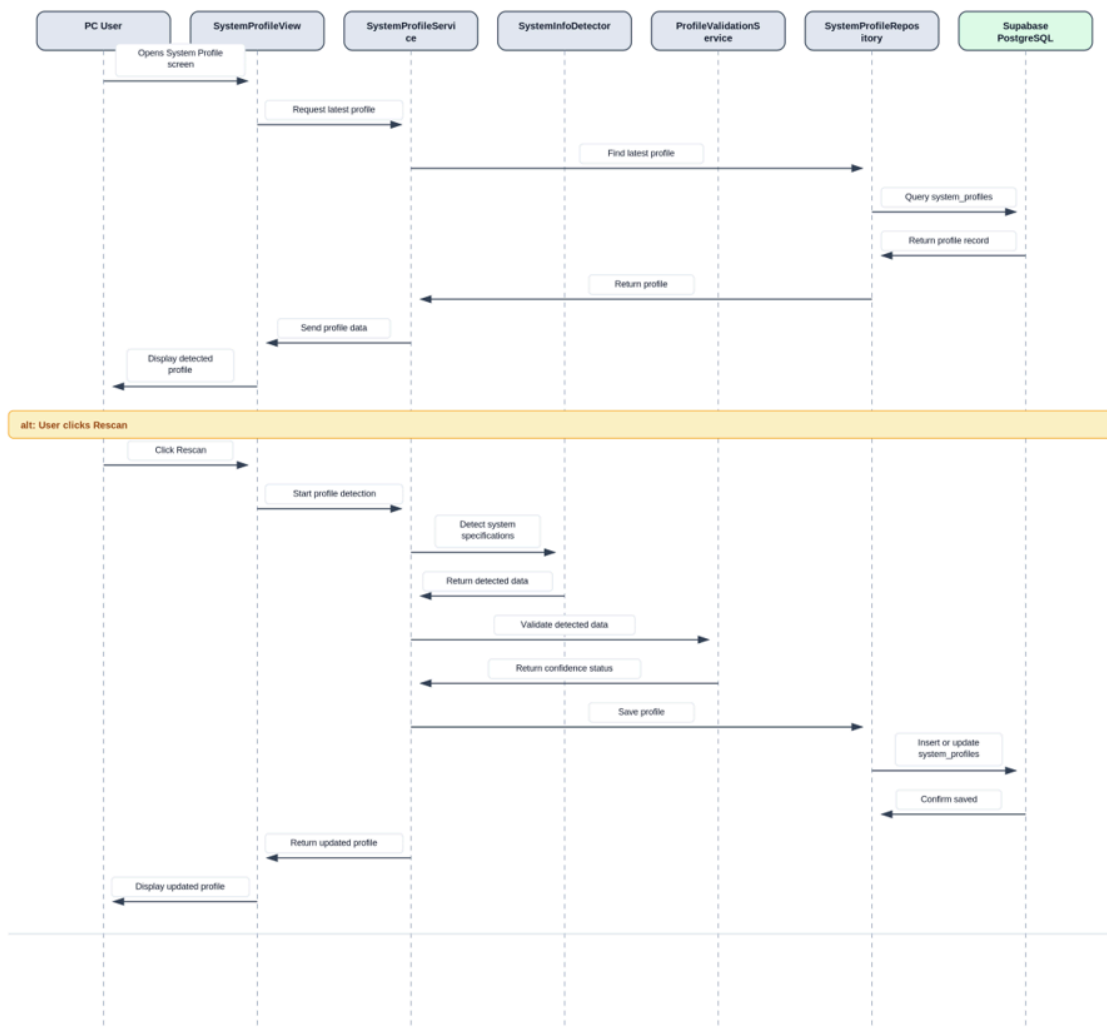
Component Name	Description and Purpose	Component Type / Format
SystemProfileService	Manages system profile detection, validation, and saving.	Python Service Class
SystemInfoDetector	Reads system information using WMI, Python platform data, or hardware information libraries.	Python Utility / Detector Class
ProfileValidationService	Checks incomplete, missing, or low-confidence profile data before saving.	Python Service Class
SystemProfileRepository	Saves and retrieves system profile records from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class

Object-Oriented Components

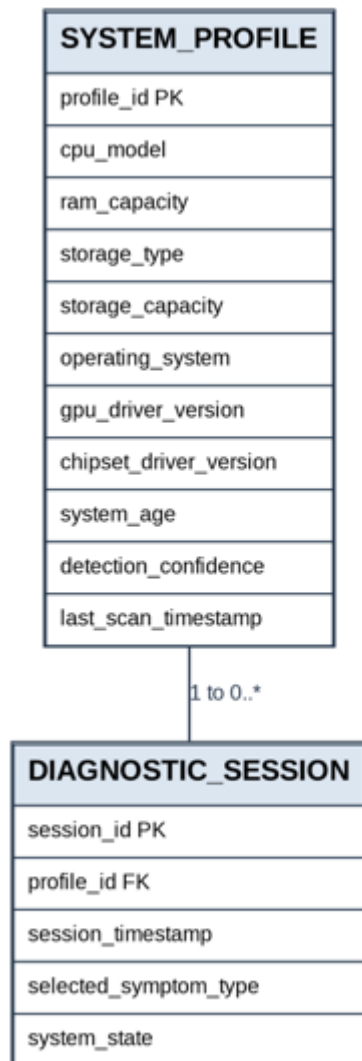
Module 1.2: System Profile Detection - Class Diagram



Module 1.2: System Profile Detection - Sequence Diagram



Data Design / ERD or Schema for System Profile Detection



Module 2 - Structured OS Symptom Intake Module

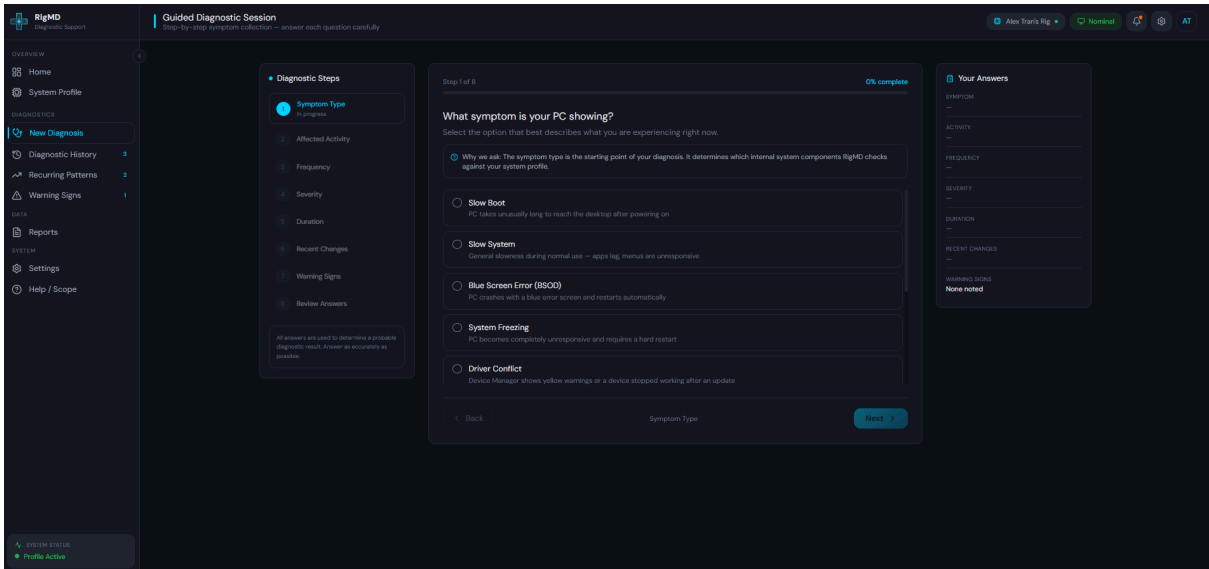
This module guides the user through controlled diagnostic questions. It collects consistent Windows-observable symptom data without requiring the user to write free-text explanations or determine which technical details are relevant.

2.1 Guided Diagnostic Intake

User Interface Design

The Guided Diagnostic Intake screen presents a step-by-step questionnaire for one diagnostic session. The layout should include a diagnostic step list, progress indicator, current question card, explanation of why the question is being asked, selectable answer choices, navigation

controls, and a running summary of the user's answers. The screen supports structured collection of symptom type, affected activity, frequency, severity, duration, recent changes, warning signs, and final review.



Front-end Components for Guided Diagnostic Intake

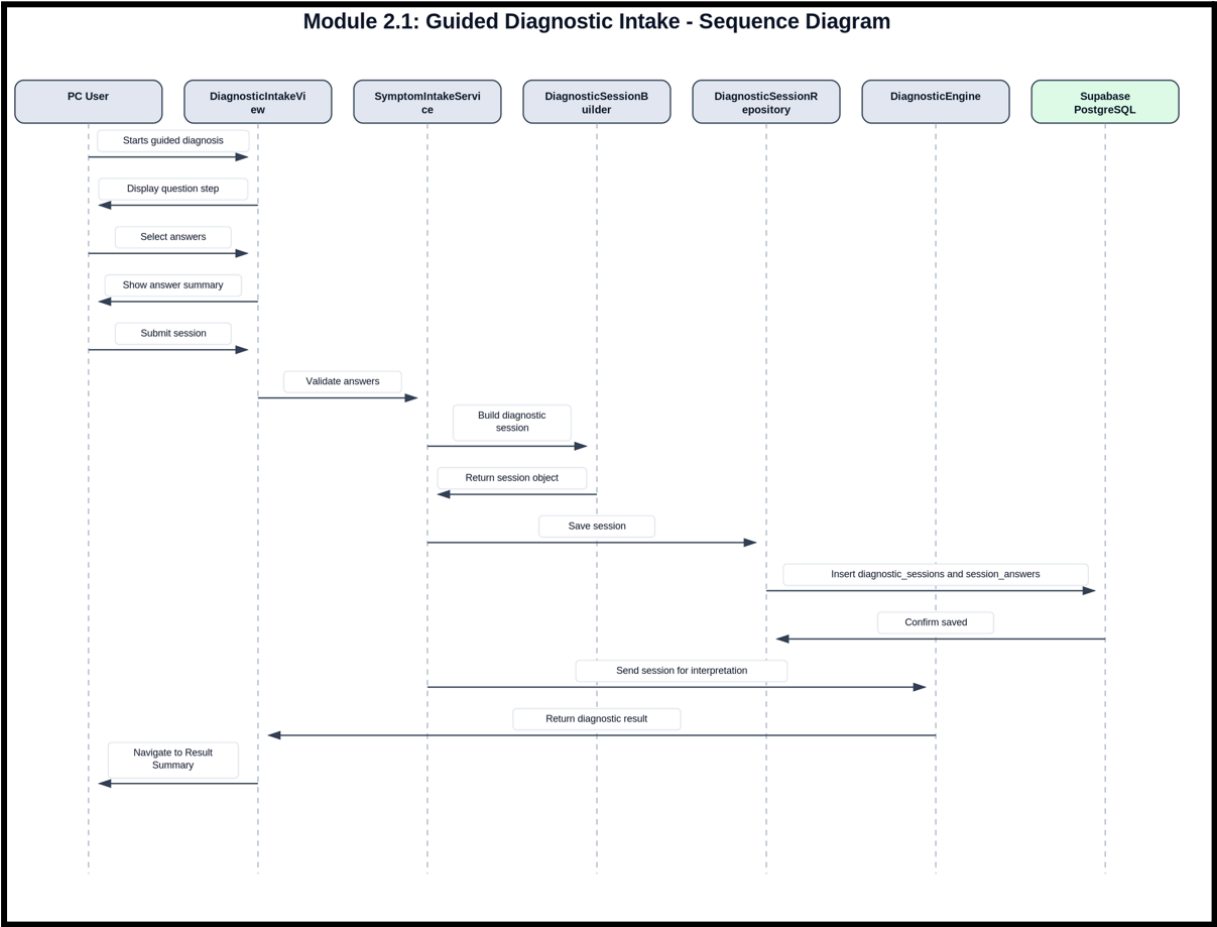
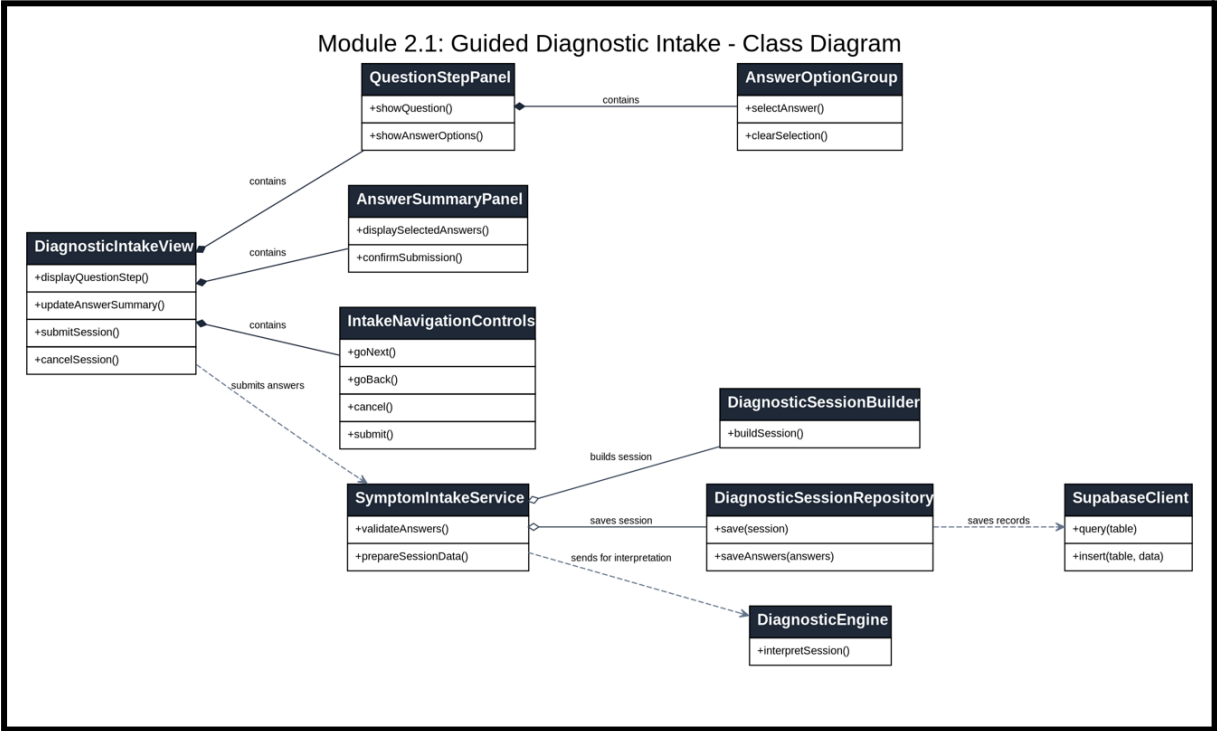
Component Name	Description and Purpose	Component Type / Format
DiagnosticIntakeView	Displays the guided diagnostic questionnaire.	CustomTkinter Screen / Frame
QuestionStepPanel	Shows one controlled question step at a time.	CustomTkinter Panel
AnswerOptionGroup	Displays selectable answer options for each question.	CustomTkinter Radio Button / Button Group
AnswerSummaryPanel	Shows the user's selected answers before final submission.	CustomTkinter Card

IntakeNavigationControls	Provides Back, Next, Cancel, and Submit controls.	CustomTkinter Button Group
--------------------------	---	----------------------------

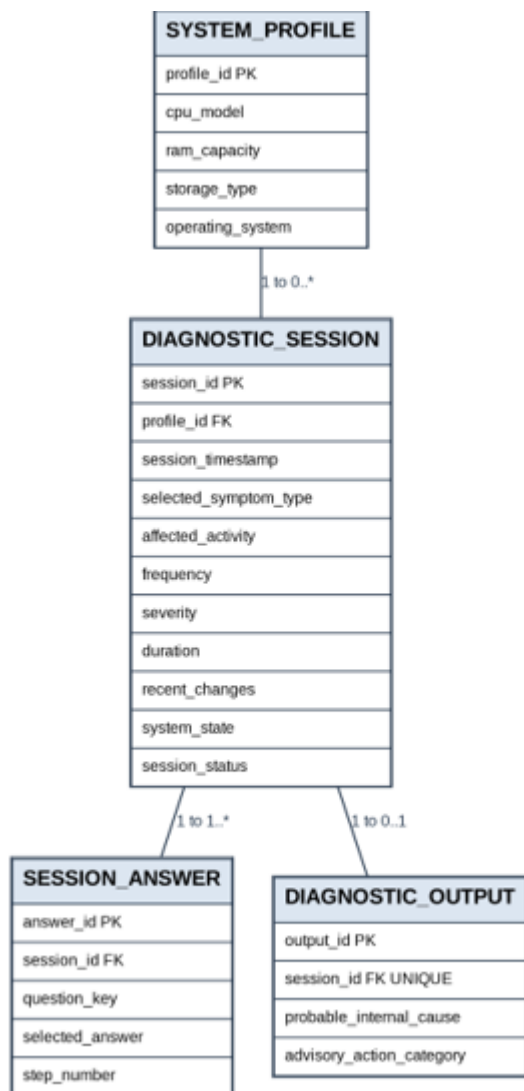
Back-end Components for Guided Diagnostic Intake

Component Name	Description and Purpose	Component Type / Format
SymptomIntakeService	Validates structured symptom answers before processing.	Python Service Class
DiagnosticSessionBuilder	Builds the complete diagnostic session record from user answers.	Python Builder Class
DiagnosticSessionRepository	Saves diagnostic session records to Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
DiagnosticEngine	Receives structured intake data and prepares it for diagnostic interpretation.	Python Engine Class

Object-Oriented Components



Data Design / ERD or Schema for Guided Diagnostic Intake



Module 3 - Internal Diagnostic Interpretation Engine

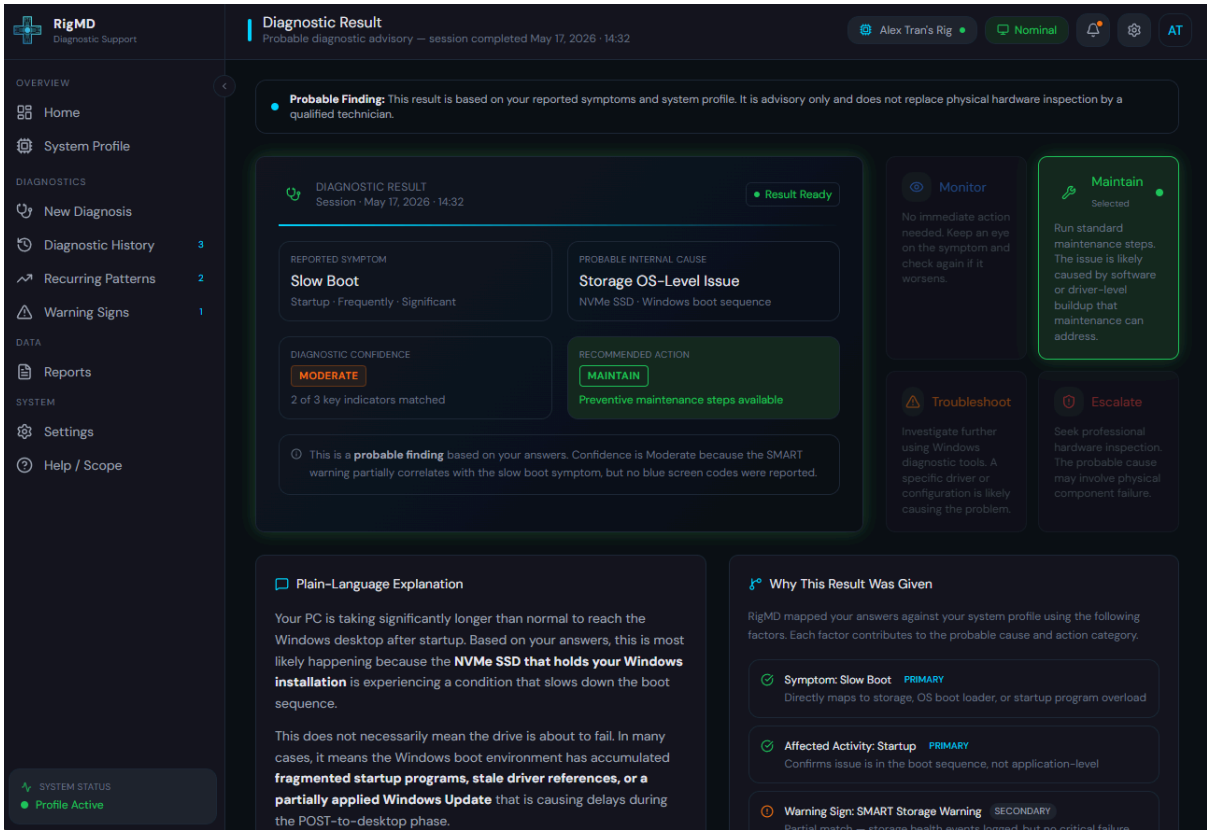
This module represents the result interpretation stage after the guided intake is submitted. The diagnostic result is shown as a scrollable interface with result summary, supporting diagnostic details, recommended next steps, and decision confidence information.

3.1 Diagnostic Result Summary

User Interface Design

The Diagnostic Result Summary section appears at the top of the completed diagnostic result screen. It presents the advisory notice, reported symptom, probable internal cause, diagnostic

confidence, recommended action category, and selected action card. It also introduces the plain-language explanation and reasoning section so the user can quickly understand what the system found and how certain the result is.



Front-end Components for Diagnostic Result Summary

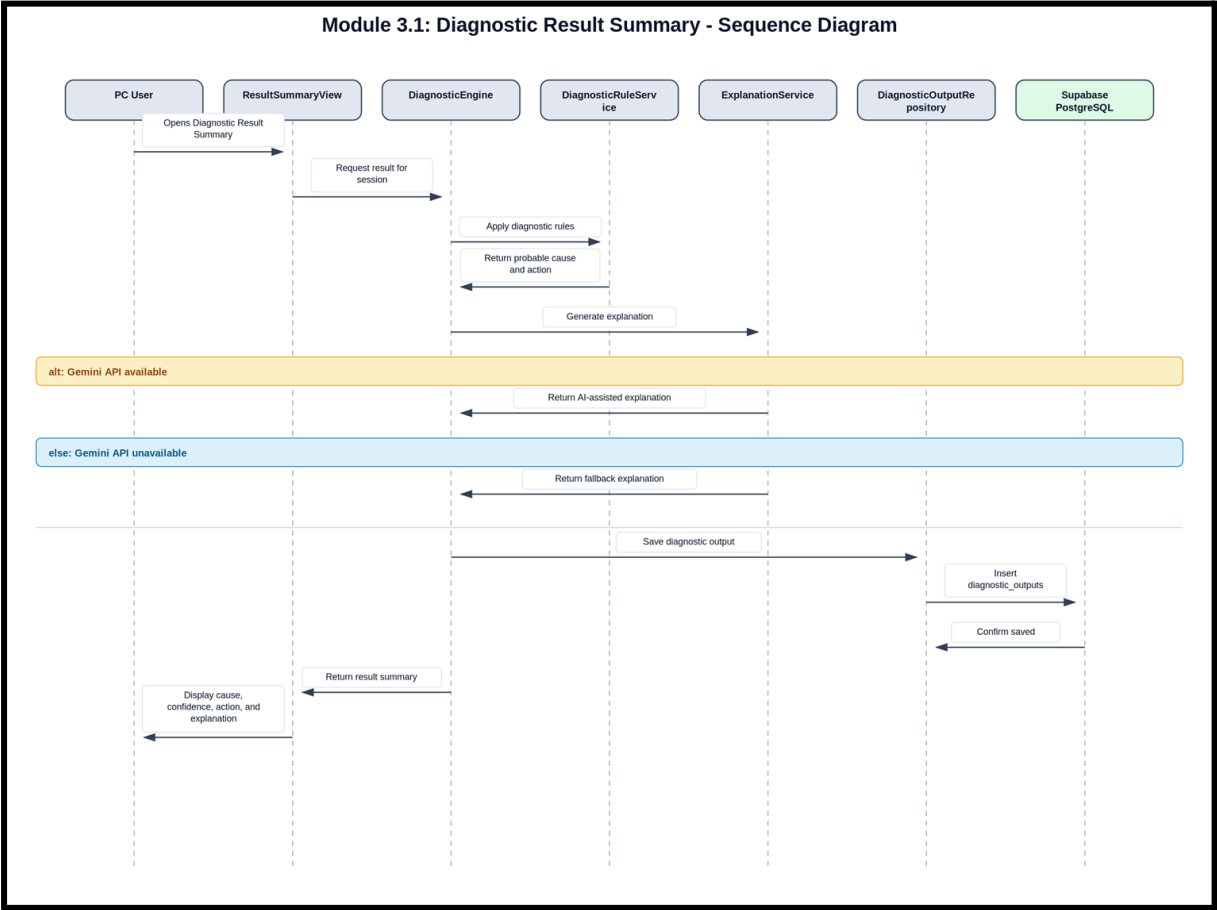
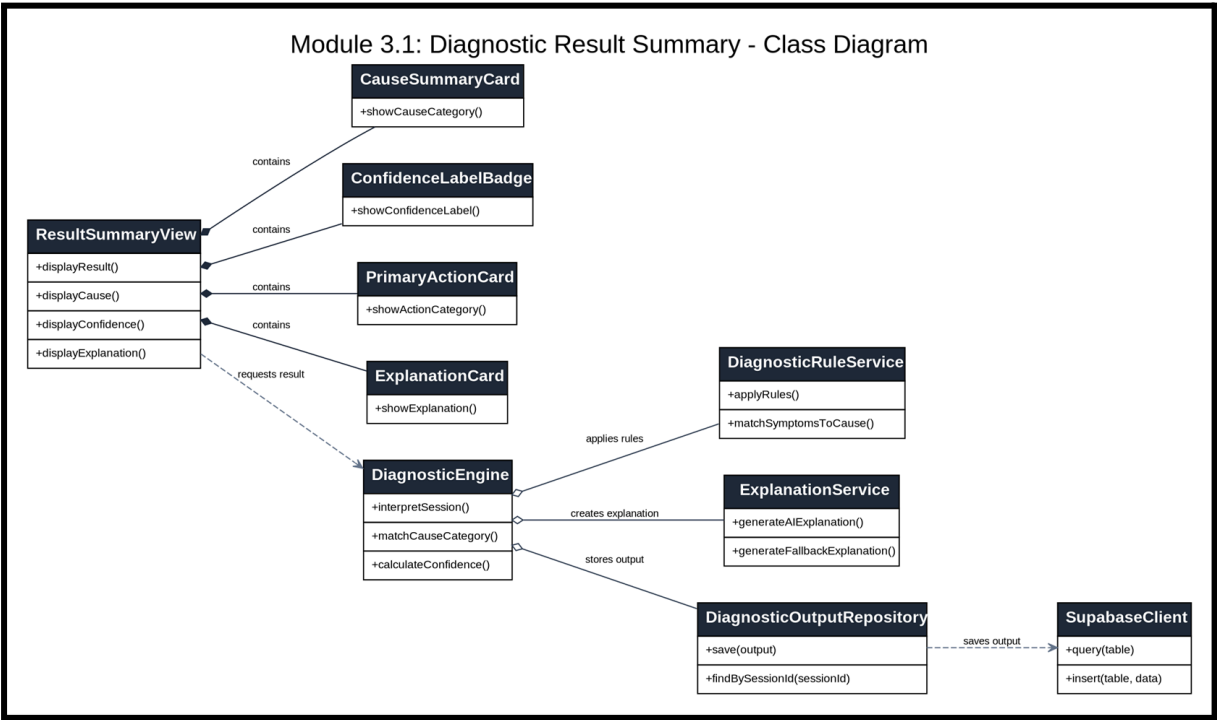
Component Name	Description and Purpose	Component Type / Format
ResultSummaryView	Displays the probable internal cause, confidence label, and primary recommended action.	CustomTkinter Screen / Frame
CauseSummaryCard	Shows the most likely internal cause category.	CustomTkinter Card

ConfidenceLabelBadge	Displays the diagnostic confidence level.	CustomTkinter Badge / Label
PrimaryActionCard	Displays the main action category such as Monitor, Maintain, Troubleshoot, or Escalate.	CustomTkinter Card
ExplanationCard	Shows the plain-language explanation for the result.	CustomTkinter Card

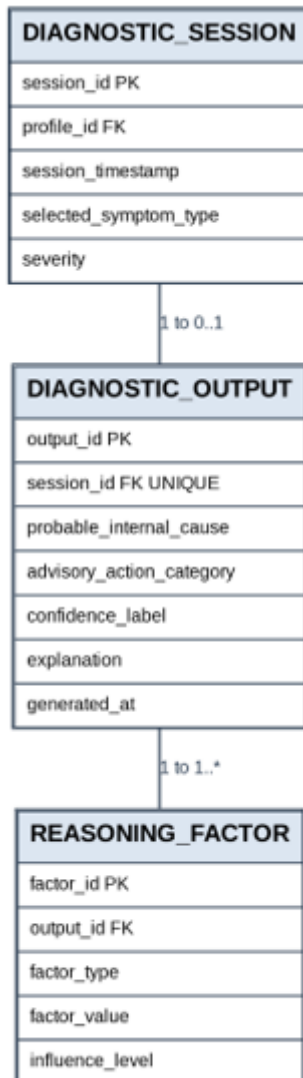
Back-end Components for Diagnostic Result Summary

Component Name	Description and Purpose	Component Type / Format
DiagnosticEngine	Maps symptom answers, system profile data, and history into a probable cause category.	Python Engine Class
DiagnosticRuleService	Applies structured diagnostic rules to classify the result.	Python Service Class
ExplanationService	Generates AI-assisted or fallback rule-based explanation text.	Python Service Class
DiagnosticOutputRepository	Saves diagnostic result records to Supabase PostgreSQL.	Supabase PostgreSQL Repository Class

Object-Oriented Components



Data Design / ERD or Schema for Diagnostic Result Summary

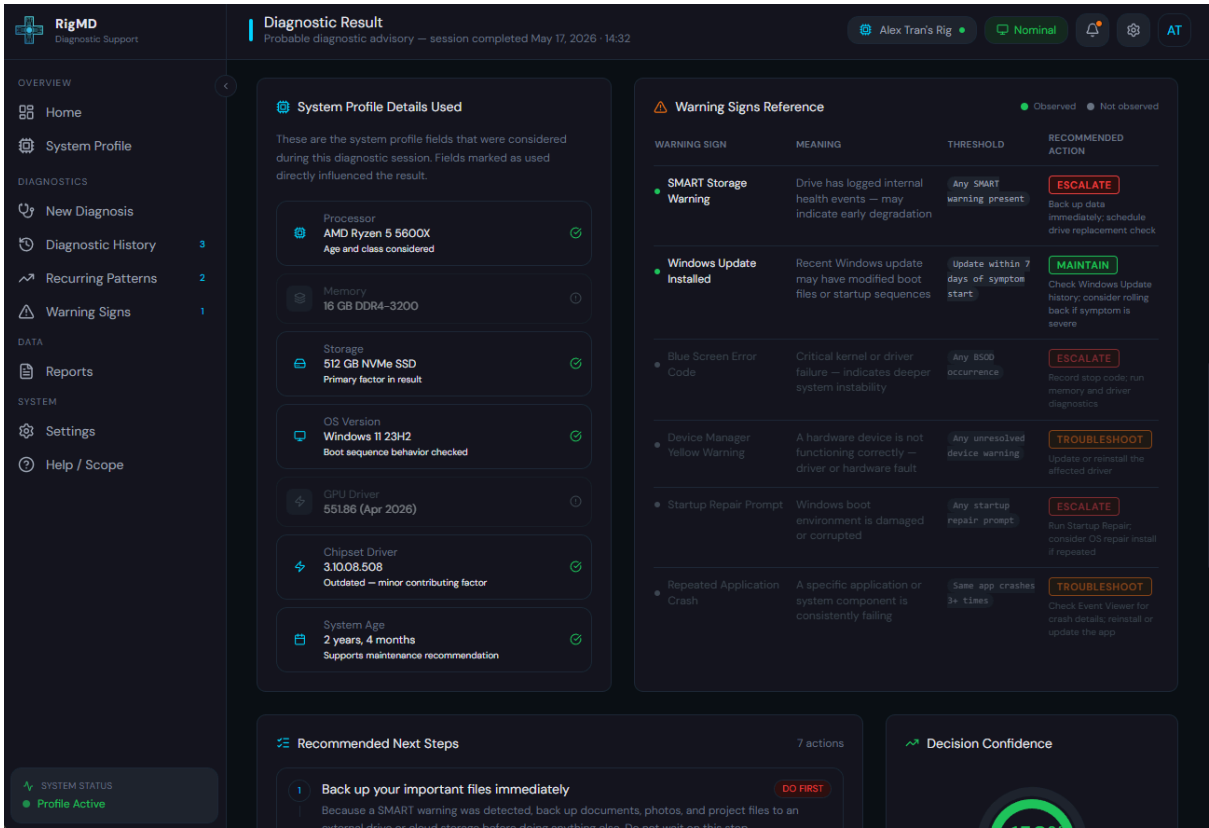


3.2 Supporting Diagnostic Details

User Interface Design

The Supporting Diagnostic Details section explains the evidence behind the result. It shows the system profile fields considered during the session, such as processor, memory, storage, operating system version, GPU driver, chipset driver, and system age. It also displays the

warning signs reference table, including warning sign, meaning, threshold, and recommended action.



Front-end Components for Supporting Diagnostic Details

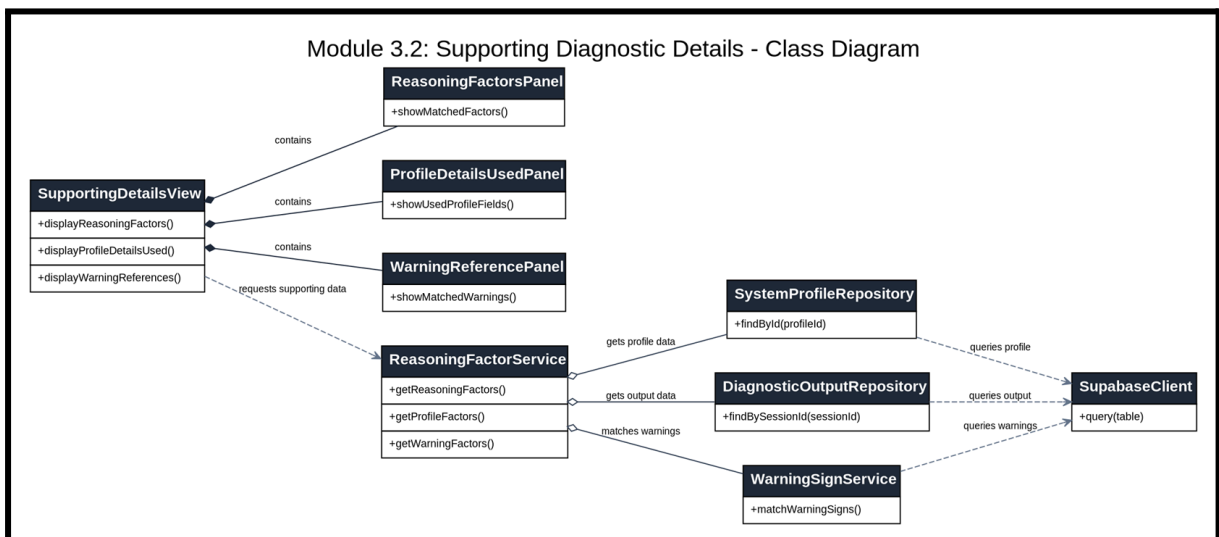
Component Name	Description and Purpose	Component Type / Format
SupportingDetailsView	Displays the supporting details used in the diagnostic result.	CustomTkinter Screen / Frame
ReasoningFactorsPanel	Shows the exact profile fields and symptom answers that influenced the result.	CustomTkinter Card
ProfileDetailsUsedPanel	Displays the system profile values used by the diagnostic engine.	CustomTkinter Panel

WarningReferencePanel	Shows matched warning signs, meanings, and thresholds.	CustomTkinter Panel
-----------------------	--	---------------------

Back-end Components for Supporting Diagnostic Details

Component Name	Description and Purpose	Component Type / Format
ReasoningFactorService	Retrieves matched system profile fields, symptoms, and rule factors.	Python Service Class
SystemProfileRepository	Retrieves stored system profile data from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
DiagnosticOutputRepository	Retrieves saved diagnostic output details.	Supabase PostgreSQL Repository Class
WarningSignService	Provides warning signs connected to the diagnostic result.	Python Service Class

Object-Oriented Components



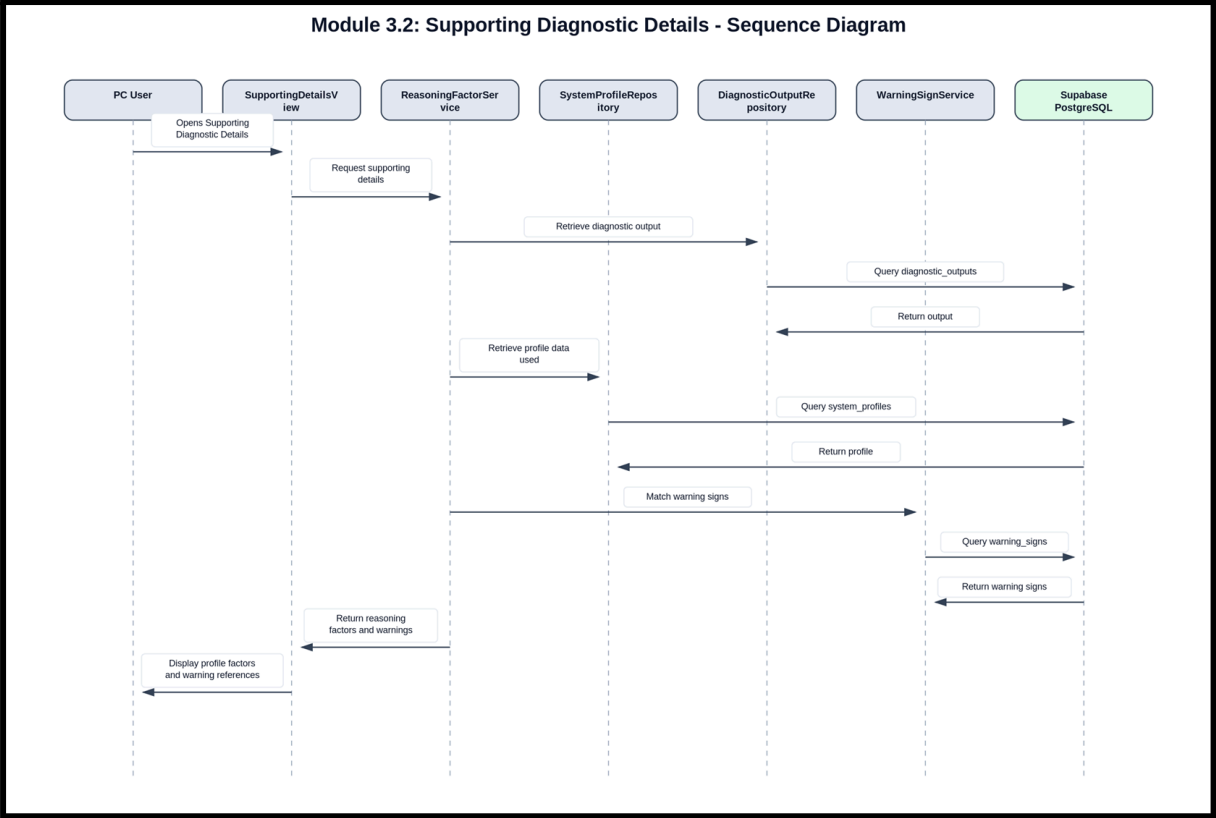
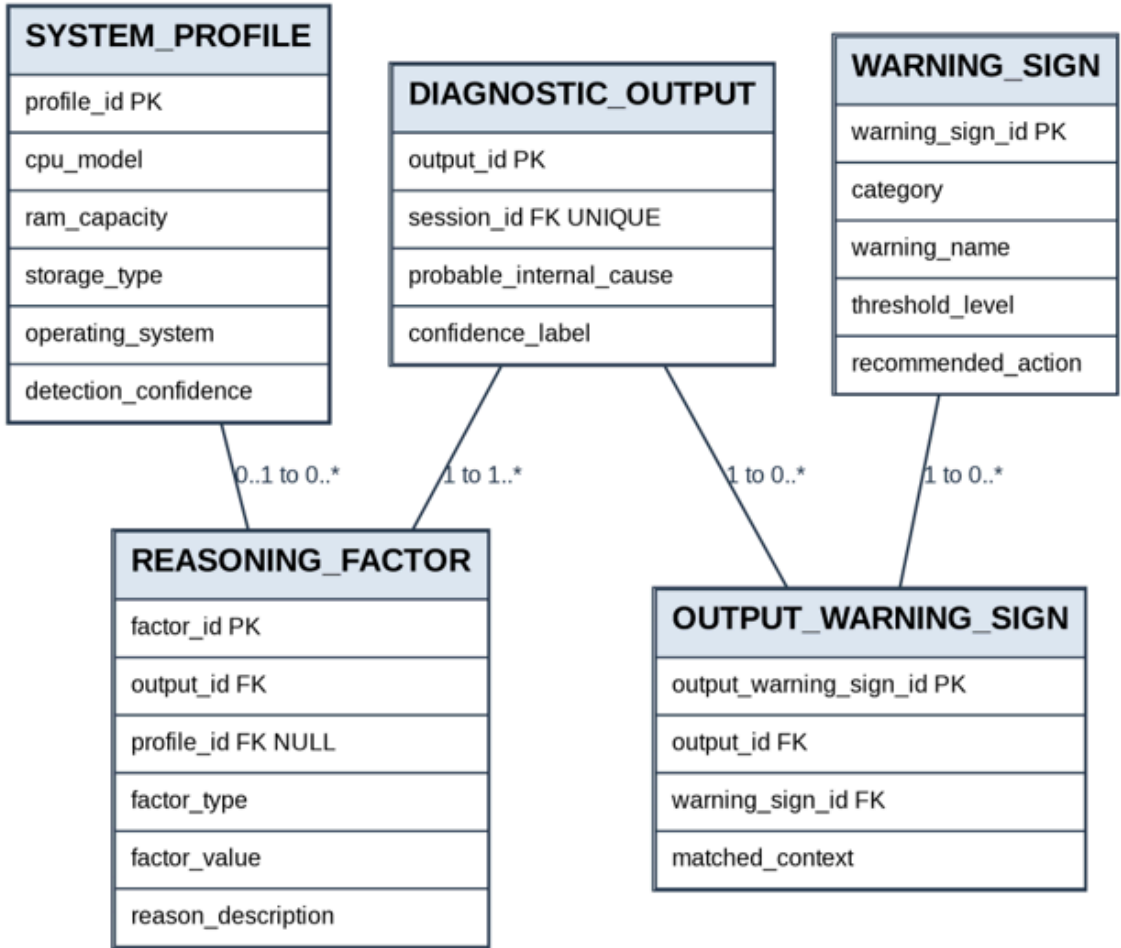


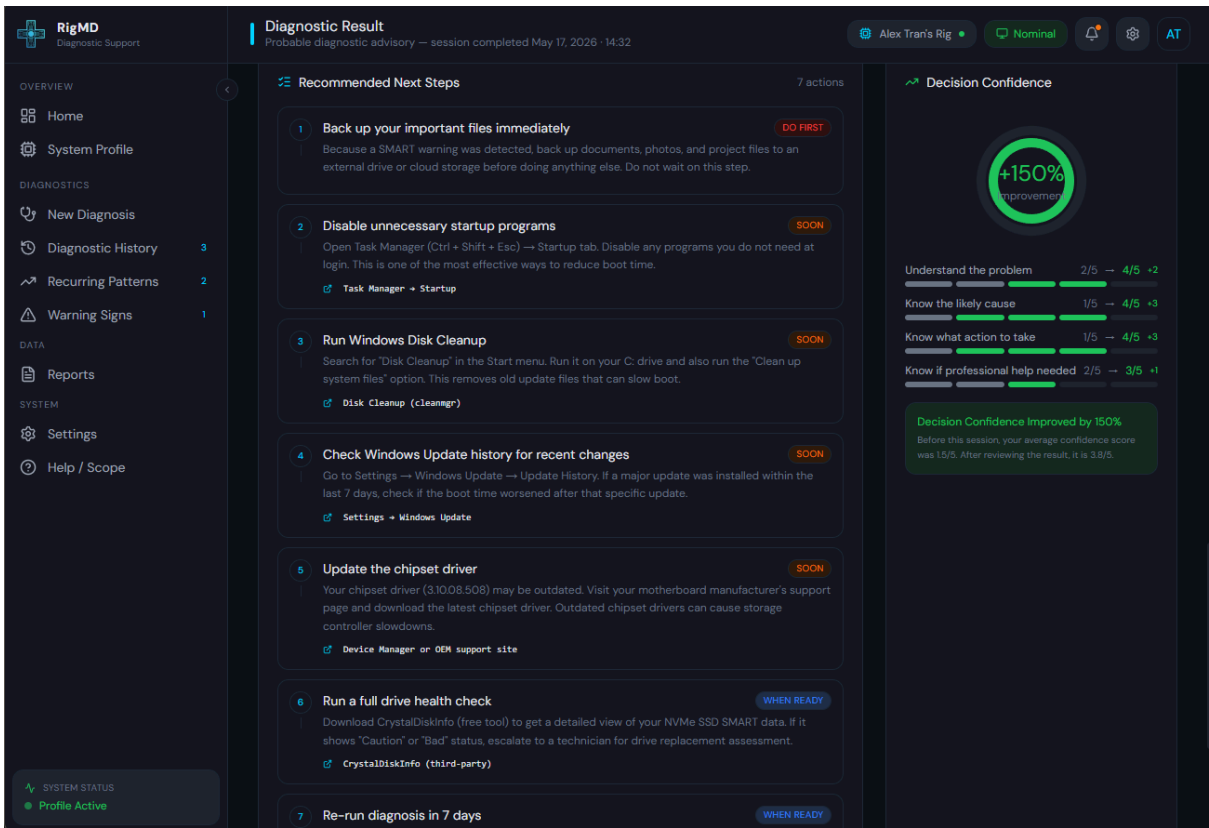
Table 3.3.2C. Data Design / ERD or Schema for Supporting Diagnostic Detail



3.3 Recommended Actions and Decision Confidence

User Interface Design

The Recommended Actions and Decision Confidence section shows the lower part of the diagnostic result screen. It lists the user's next steps in priority order and uses labels such as Do First, Soon, and When Ready. It also includes a decision confidence panel showing how the session helps the user understand the problem, identify the likely cause, choose an action, and decide whether professional help is needed.



Front-end Components for Recommended Actions and Decision Confidence

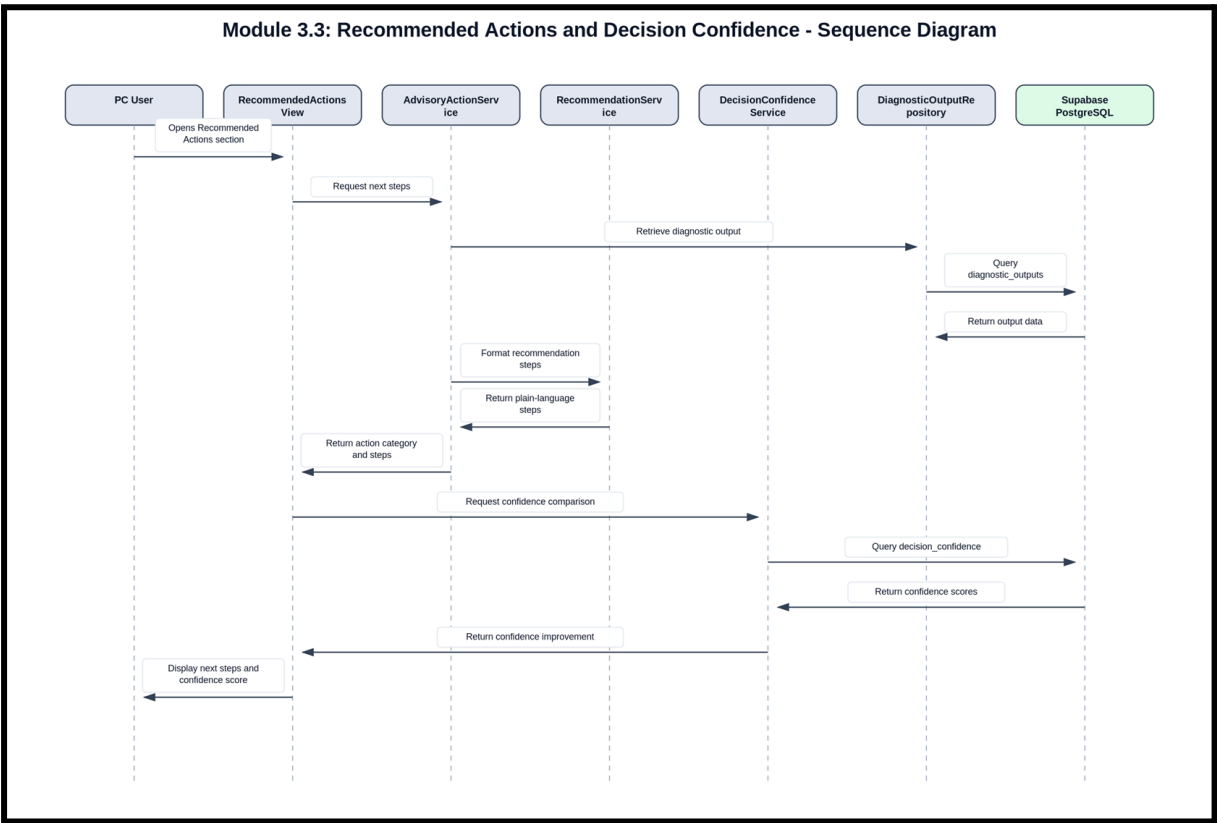
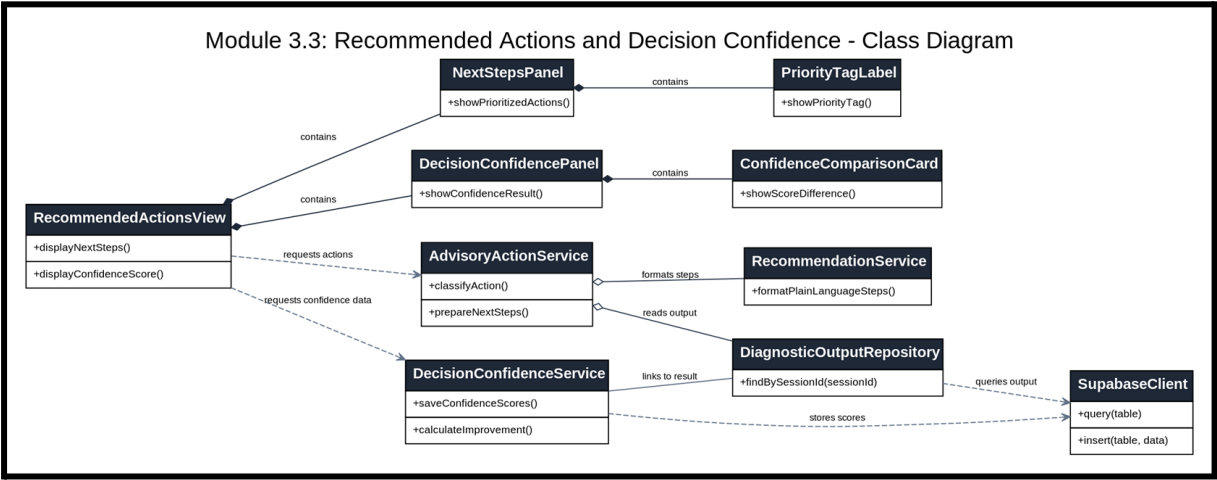
Component Name	Description and Purpose	Component Type / Format
RecommendedActionsView	Displays the recommended next steps for the diagnostic result.	CustomTkinter Screen / Frame

NextStepsPanel	Shows prioritized actions the user should follow.	CustomTkinter Panel
PriorityTagLabel	Displays labels such as “DO FIRST” or “OBSERVE”.	CustomTkinter Label / Badge
DecisionConfidencePanel	Displays the user’s decision confidence rating result.	CustomTkinter Card
ConfidenceComparisonCard	Shows the difference between pre-session and post-session confidence ratings.	CustomTkinter Card

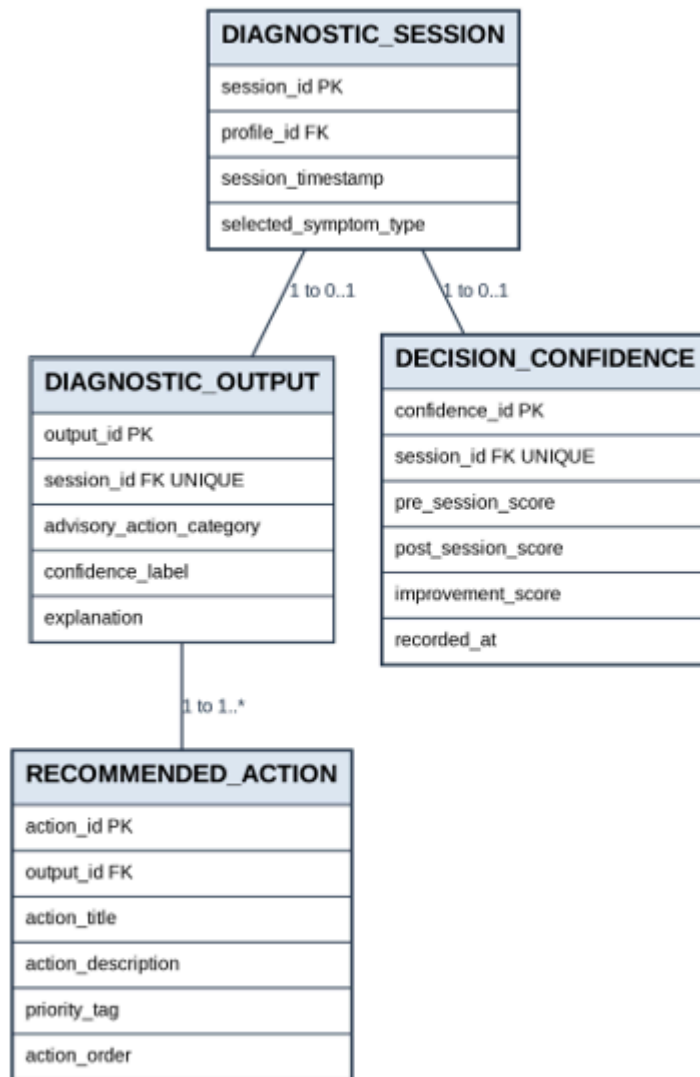
Back-end Components for Recommended Actions and Decision Confidence

Component Name	Description and Purpose	Component Type / Format
AdvisoryActionService	Assigns the correct action category and prepares recommended next steps.	Python Service Class
RecommendationService	Formats the next-step instructions in plain language.	Python Service Class
DecisionConfidenceService	Stores and compares user confidence ratings.	Python Service Class
DiagnosticOutputRepository	Retrieves completed diagnostic outputs from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class

Object-Oriented Components



Data Design / ERD or Schema for Recommended Actions and Decision Confidence



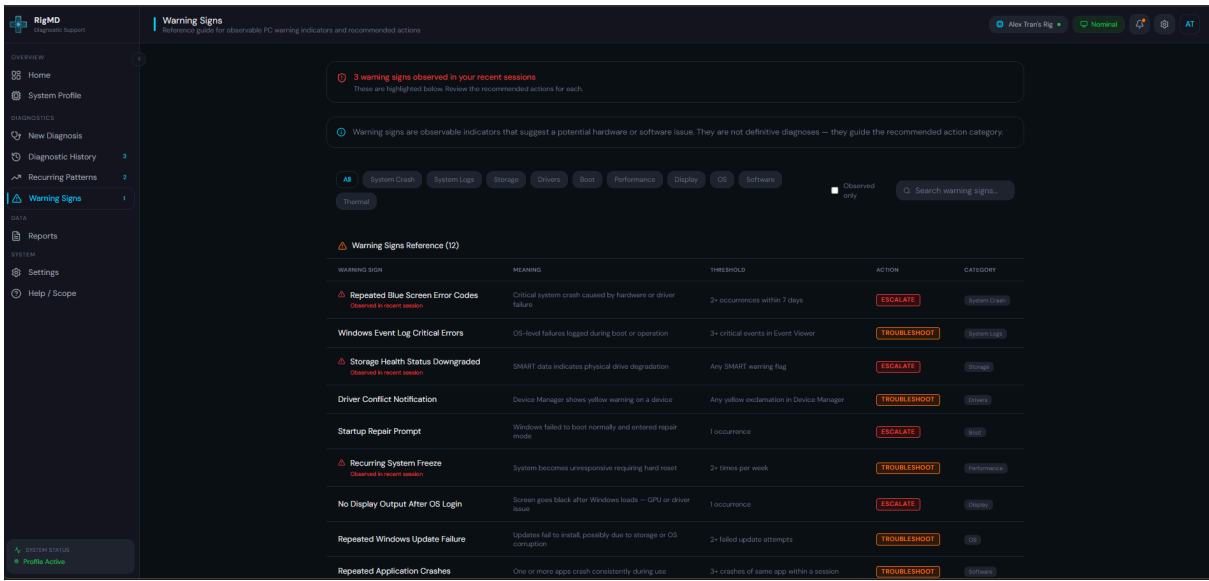
Module 4 - Advisory Action Module

This module supports the system's action-classified advisory output. It provides the warning sign reference library and helps users understand when to monitor, maintain, troubleshoot, or escalate the issue for professional inspection.

4.1 Warning Signs Reference

User Interface Design

The Warning Signs Reference screen provides a standalone guide to observable PC warning indicators. It displays active warning counts, an explanation note, category filters, an observed-only toggle, search field, and a table of warning signs. Each warning sign entry should include its meaning, threshold, recommended action, and category. This helps users recognize escalation points even outside a completed diagnostic session.



Front-end Components for Warning Signs Reference

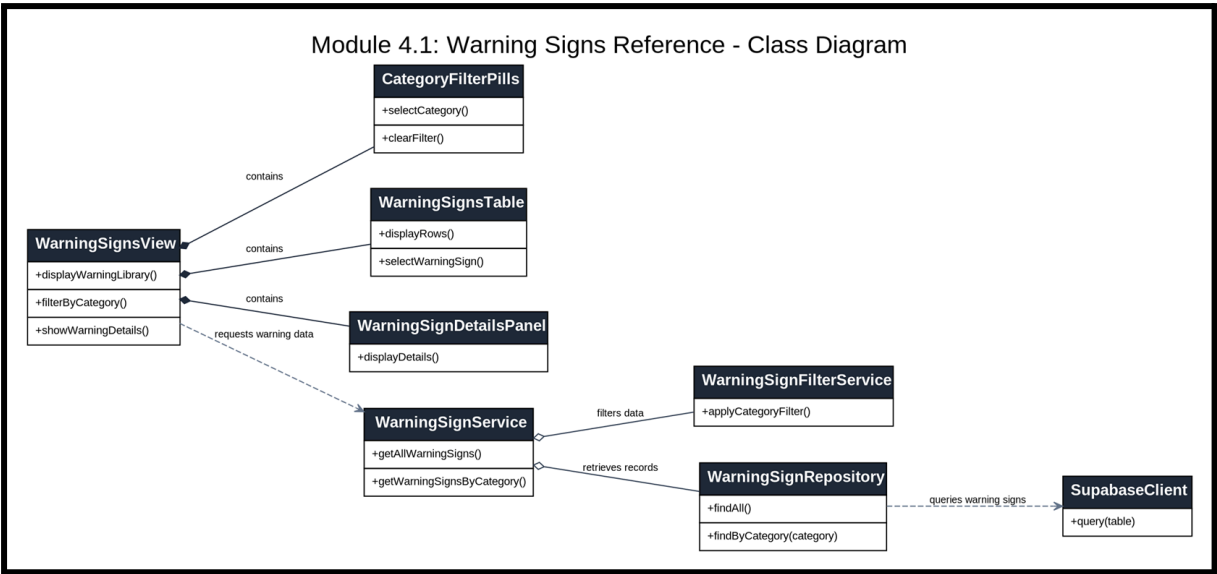
Component Name	Description and Purpose	Component Type / Format
WarningSignsView	Displays the standalone warning signs reference library.	CustomTkinter Screen / Frame
CategoryFilterPills	Allows filtering by categories such as Storage, Crash, Driver, Boot, or Performance.	CustomTkinter Filter Buttons
WarningSignsTable	Shows warning sign name, meaning, threshold, and recommended action.	CustomTkinter Table

WarningSignDetailsPanel	Displays more detailed explanation for a selected warning sign.	CustomTkinter Panel
-------------------------	---	---------------------

Back-end Components for Warning Signs Reference

Component Name	Description and Purpose	Component Type / Format
WarningSignService	Retrieves and filters warning sign reference data.	Python Service Class
WarningSignRepository	Stores and retrieves predefined warning sign records from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
WarningSignFilterService	Applies category-based filtering for warning signs.	Python Service Class

Object-Oriented Components



Module 4.1: Warning Signs Reference - Sequence Diagram

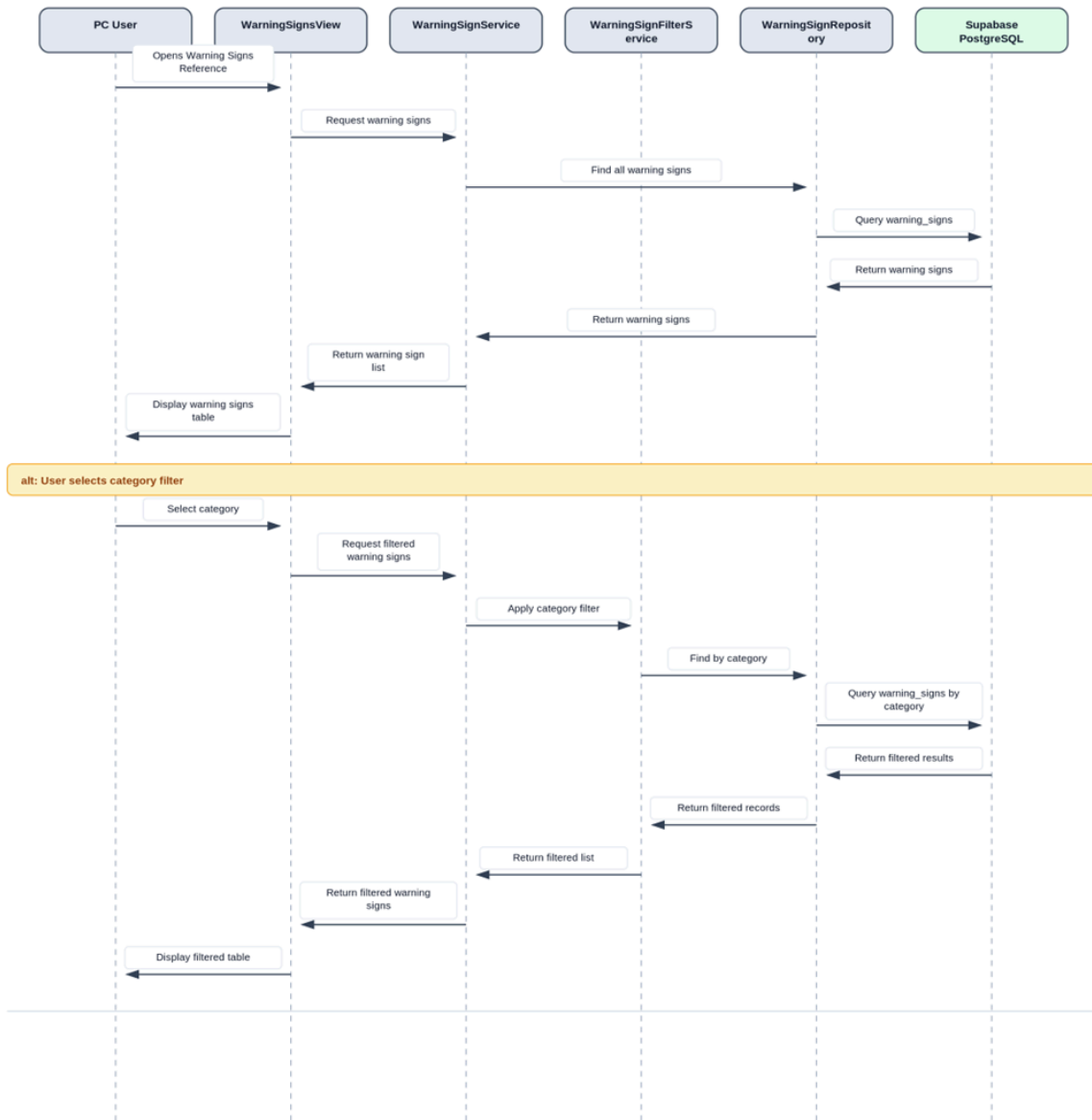
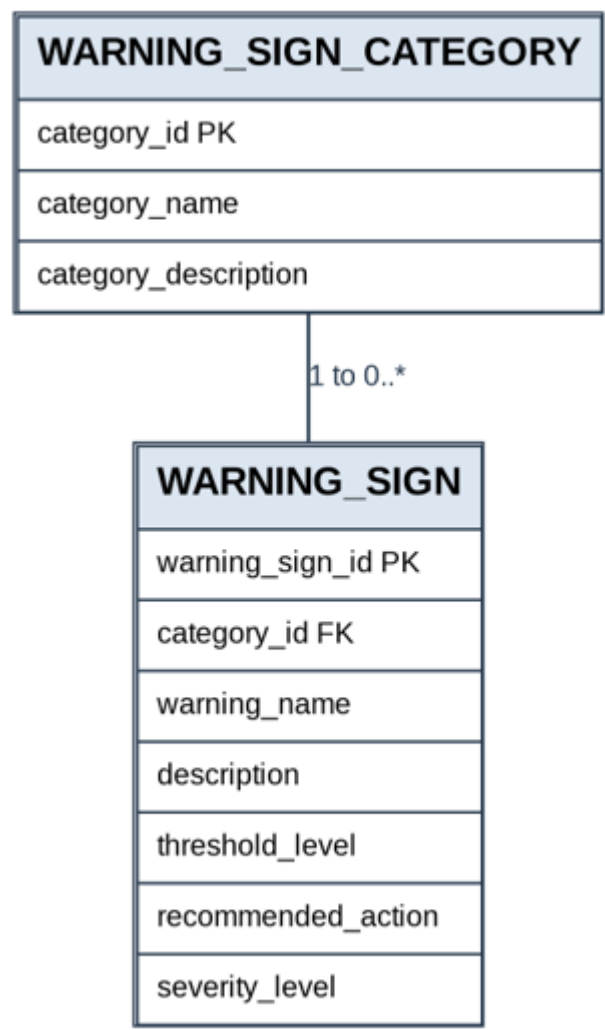


Table 3.4.1C. Data Design / ERD or Schema for Warning Signs Reference



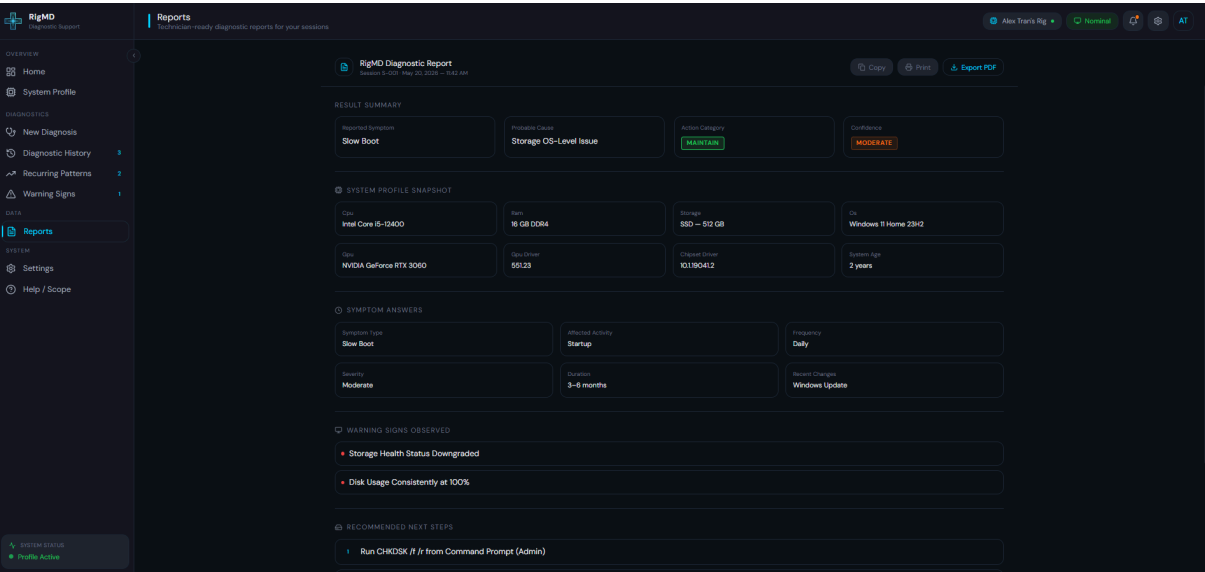
Module 5 - Transparent Result Dashboard and Report Output Module

This module supports technician-ready report output based on saved diagnostic sessions. It organizes diagnostic information into a clear report that the user can copy, print, or export.

5.1 Reports

User Interface Design

The Reports screen allows the user to select a completed diagnostic session and view a technician-ready report. The interface should include a session selector, result summary cards, system profile snapshot, symptom answers, warning signs observed, and action buttons for copy, print, and PDF export. The report supports user preparation before repair, troubleshooting, or professional consultation.



Front-end Components for Reports

Component Name	Description and Purpose	Component Type / Format
ReportsView	Allows the user to select a completed session and generate a technician-ready report.	CustomTkinter Screen / Frame
SessionDropdownSelector	Lets the user choose which diagnostic session to include in the report.	CustomTkinter Dropdown
ReportPreviewPanel	Displays the report preview before exporting.	CustomTkinter Preview Panel
ReportActionButton	Provides Copy, Print, and Export PDF actions.	CustomTkinter Button Group

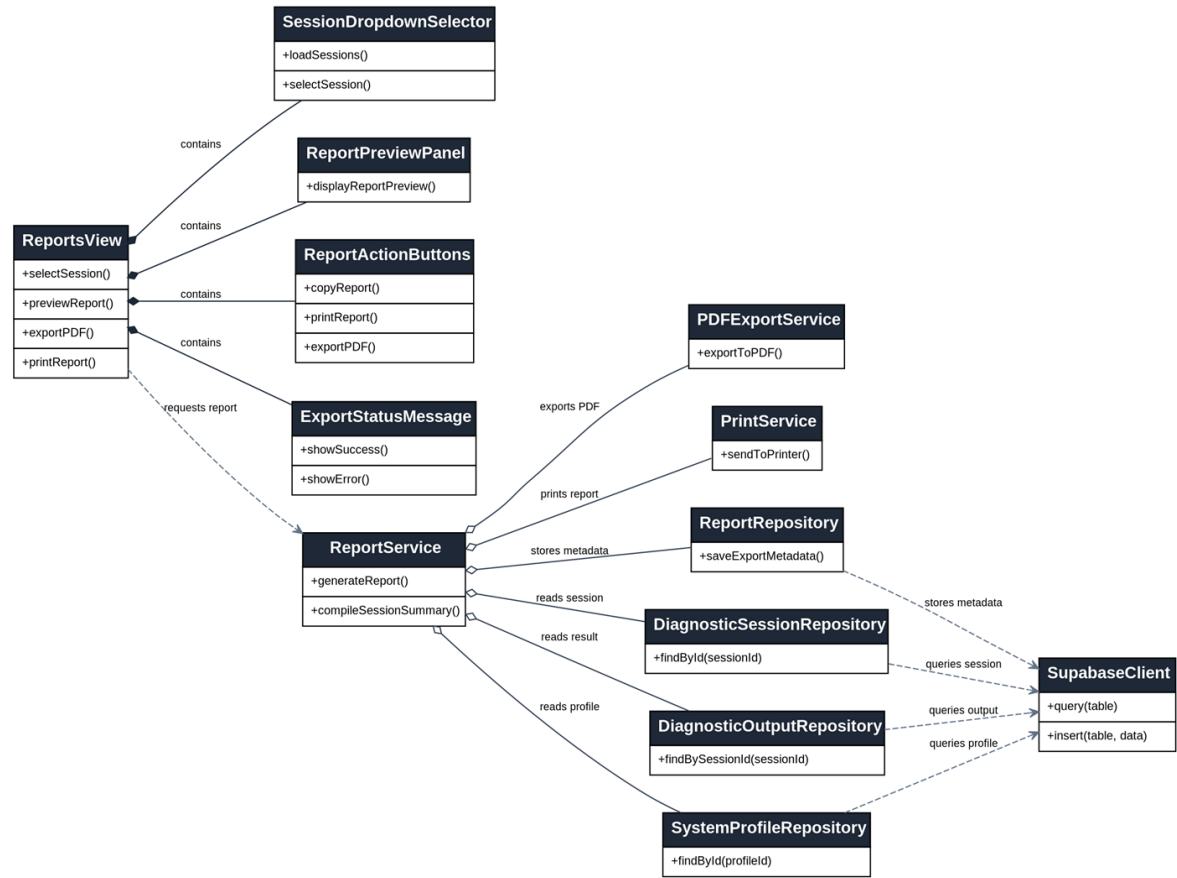
ExportStatusMessage	Shows success or error messages after report export.	CustomTkinter Label / Alert
---------------------	--	-----------------------------

Back-end Components for Reports

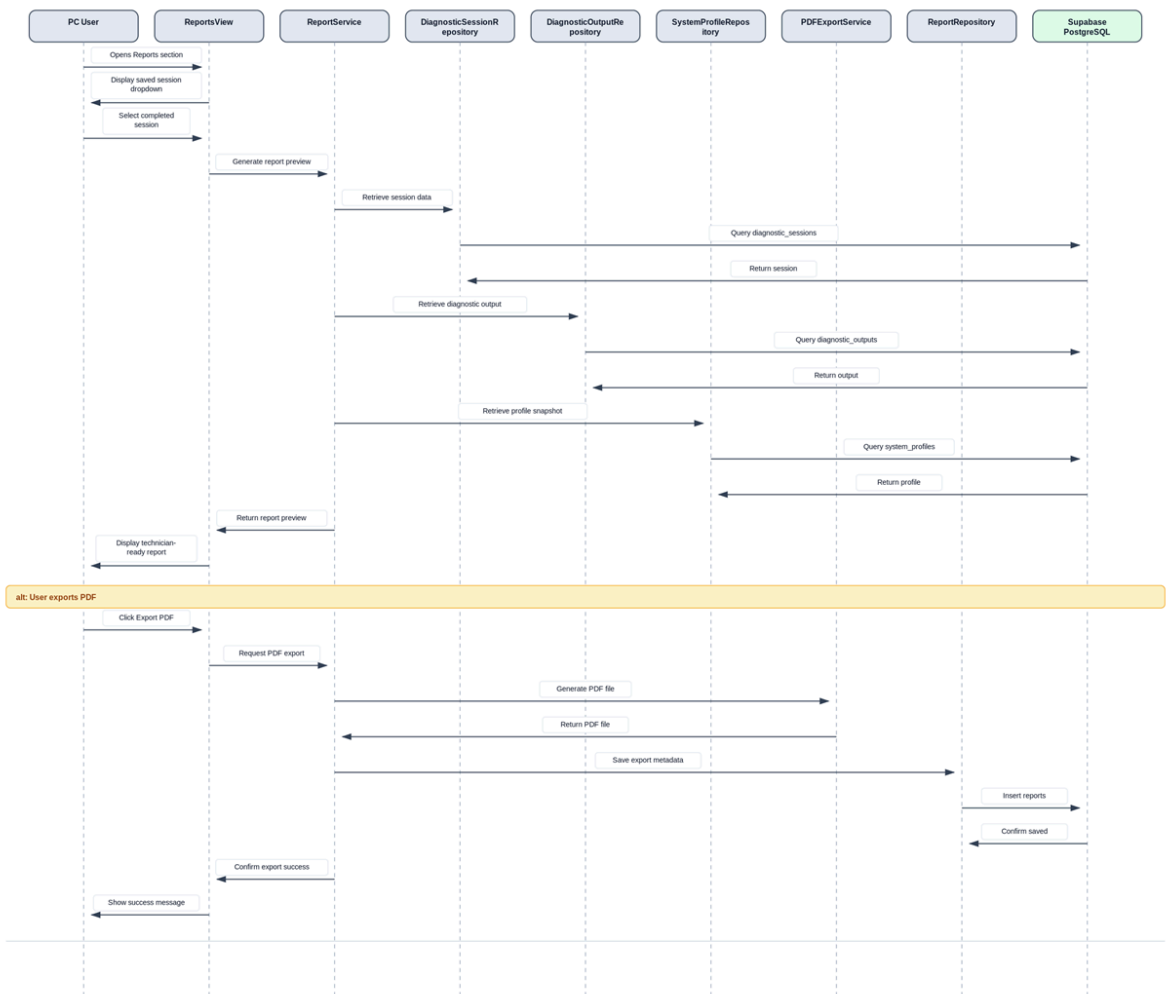
Component Name	Description and Purpose	Component Type / Format
ReportService	Compiles session data, profile snapshot, symptom answers, warning signs, and results into a report.	Python Service Class
ReportRepository	Stores report metadata in Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
DiagnosticSessionRepository	Retrieves the selected session record.	Supabase PostgreSQL Repository Class
DiagnosticOutputRepository	Retrieves the selected session's diagnostic output.	Supabase PostgreSQL Repository Class
PDFExportService	Converts the report preview into a PDF file.	Python Utility Service
PrintService	Sends the generated report to the system print function.	Python Utility Service

Object-Oriented Components

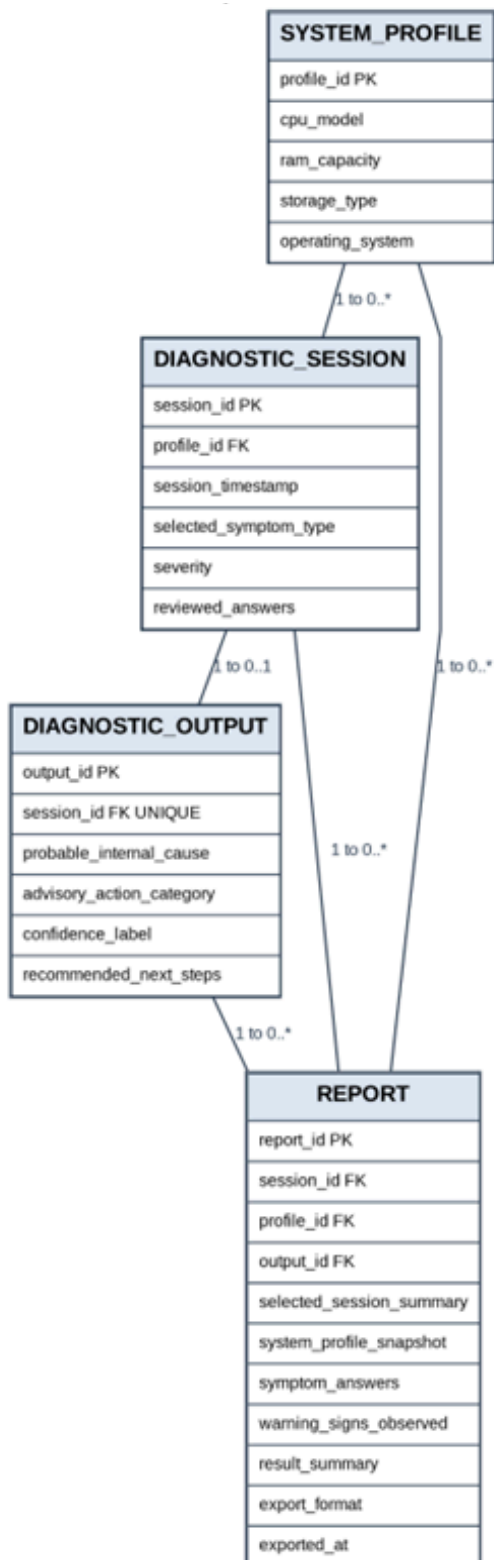
Module 5.1: Reports - Class Diagram



Module 5.1: Reports - Sequence Diagram



Data Design / ERD or Schema for Reports



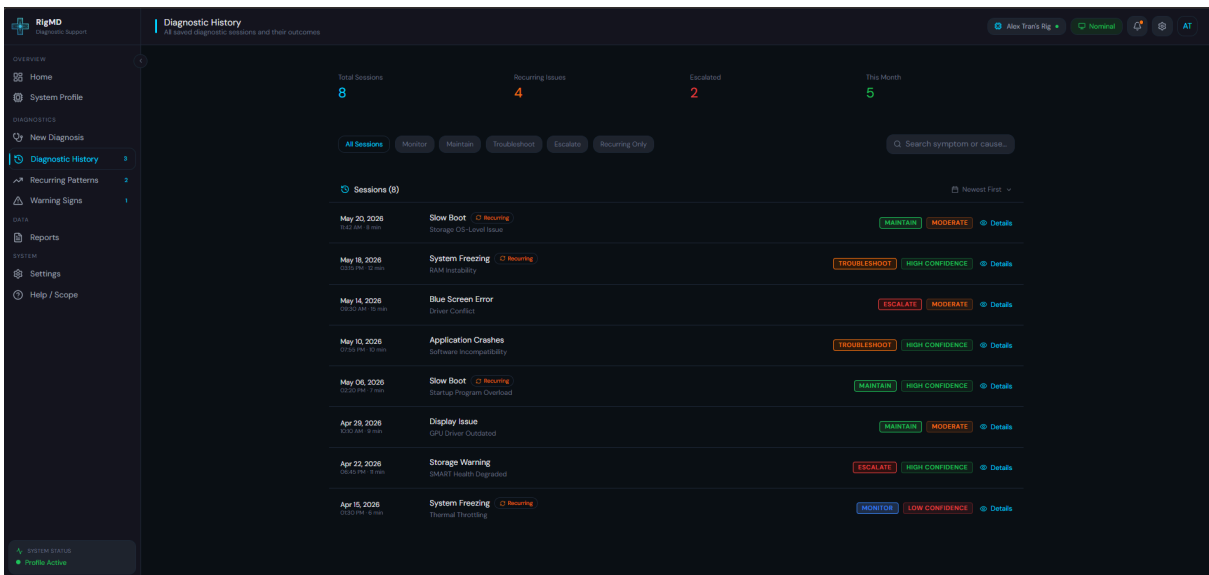
Module 6 - Session History and Recurring Pattern Detection Module

This module provides the longitudinal tracking layer of RigMD. It stores completed diagnostic sessions locally and detects repeated symptoms or internal cause categories across two or more sessions.

6.1 Diagnostic History

User Interface Design

The Diagnostic History screen lists saved diagnostic sessions in chronological order. It includes summary metrics, action category filters, search, sorting, session entries, symptom titles, probable causes, action categories, confidence labels, recurring tags, and detail links. The screen helps users review prior results and retrieve previous advisory outputs when needed.



Front-end Components for Diagnostic History

Component Name	Description and Purpose	Component Type / Format
DiagnosticHistoryView	Displays saved diagnostic sessions in chronological order.	CustomTkinter Screen / Frame

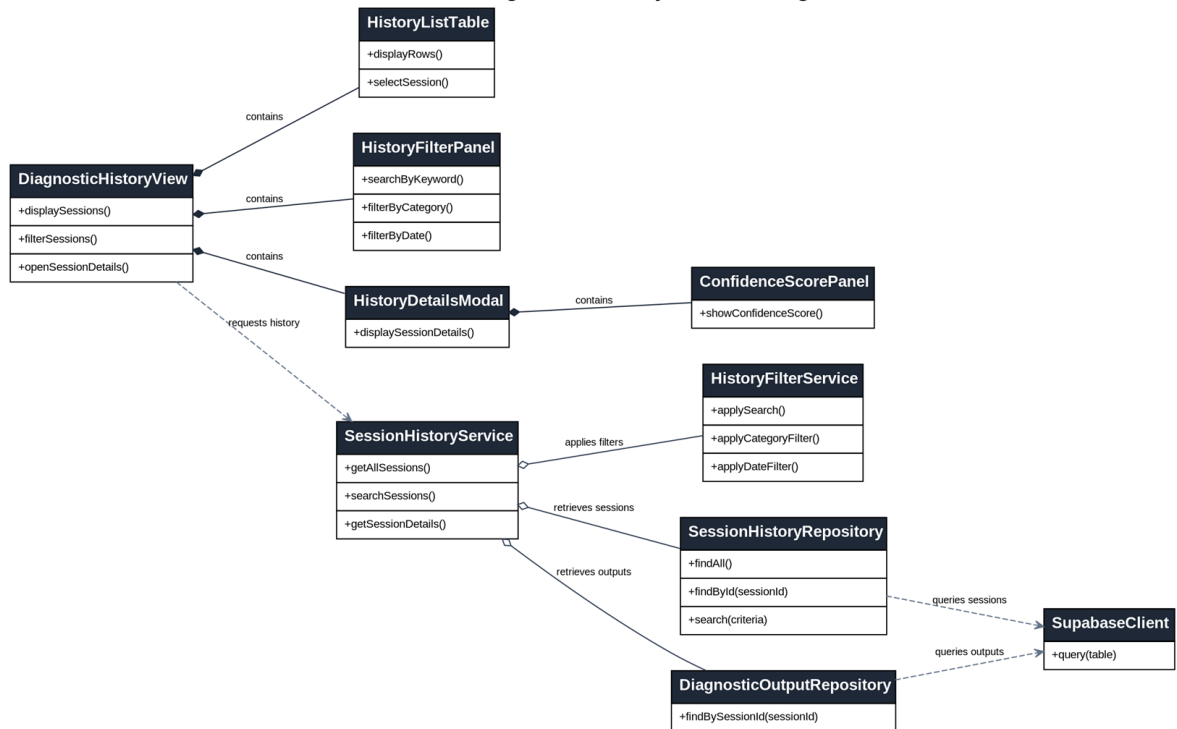
HistoryListTable	Shows previous sessions, dates, symptoms, and action outcomes.	CustomTkinter Table
HistoryFilterPanel	Allows searching and filtering by symptom, date, cause, or action category.	CustomTkinter Filter Panel
HistoryDetailsModal	Displays full details of a selected past session.	CustomTkinter Modal
ConfidenceScorePanel	Shows the confidence score connected to a past session.	CustomTkinter Card

Back-end Components for Diagnostic History

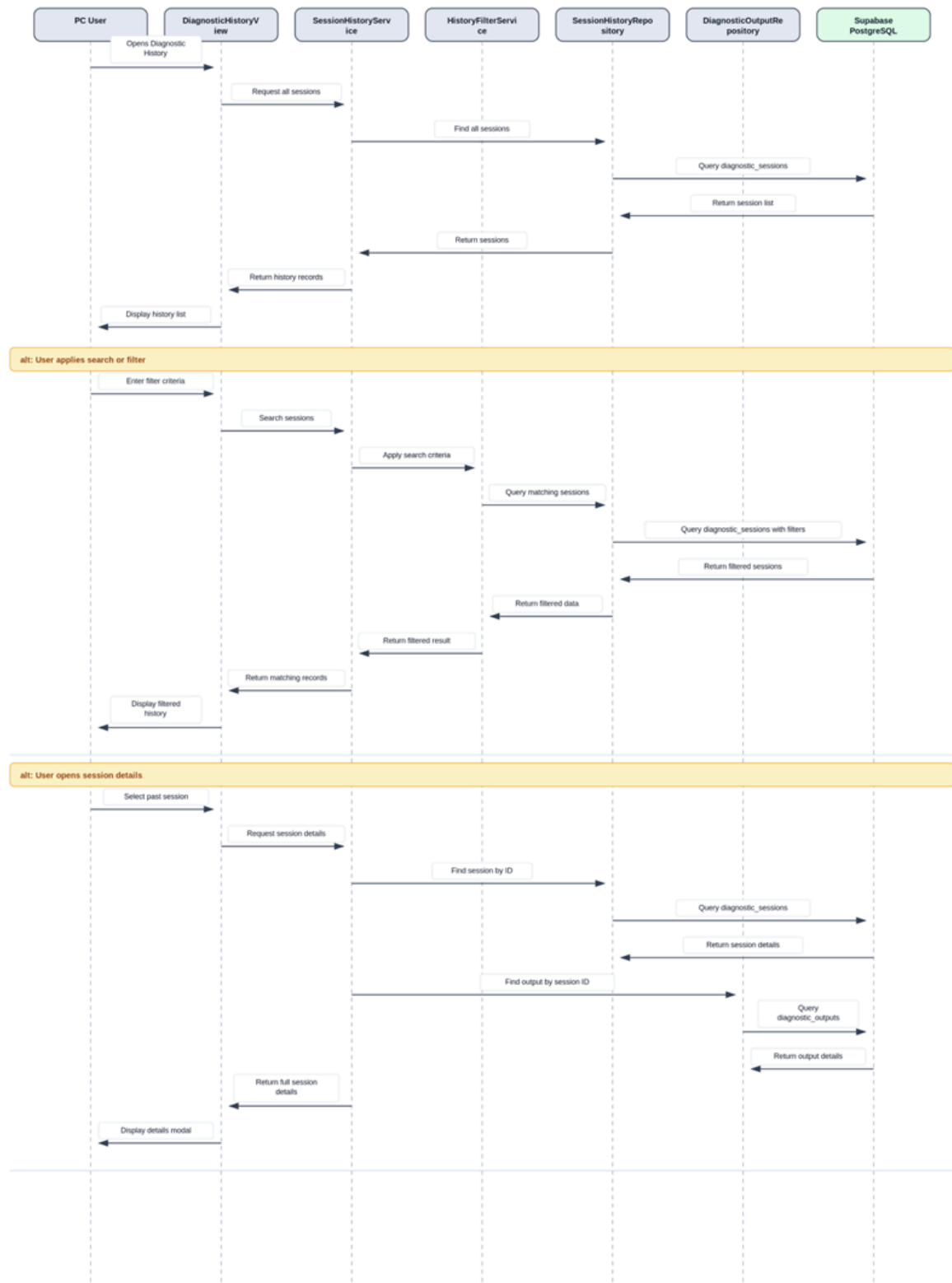
Component Name	Description and Purpose	Component Type / Format
SessionHistoryService	Retrieves, searches, and filters saved diagnostic sessions.	Python Service Class
SessionHistoryRepository	Reads diagnostic session records from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
DiagnosticOutputRepository	Retrieves result data connected to each session.	Supabase PostgreSQL Repository Class
HistoryFilterService	Applies search and filter rules to session history.	Python Service Class

Object-Oriented Components

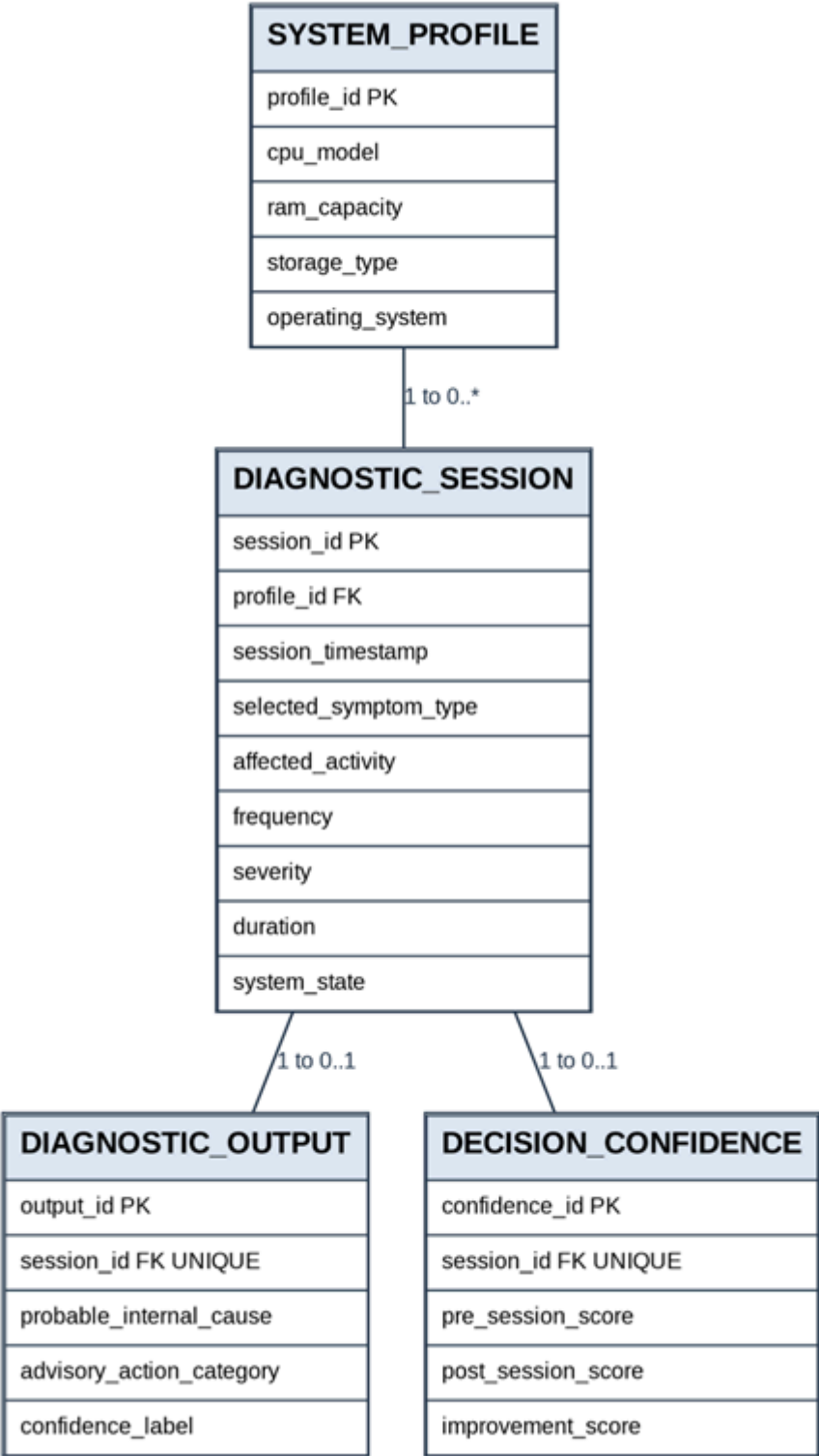
Module 6.1: Diagnostic History - Class Diagram



Module 6.1: Diagnostic History - Sequence Diagram



Data Design / ERD or Schema for Diagnostic History



6.2 Recurring Patterns

User Interface Design

The Recurring Patterns screen shows repeated symptoms and internal cause categories detected across saved diagnostic sessions. The interface should display recurring issue count, worsening trend count, action escalation count, total occurrences, repeated symptom cards, first seen date, latest date, trend label, and previous-to-current action comparison. This screen helps users identify whether an issue is stable, improving, worsening, or escalating.

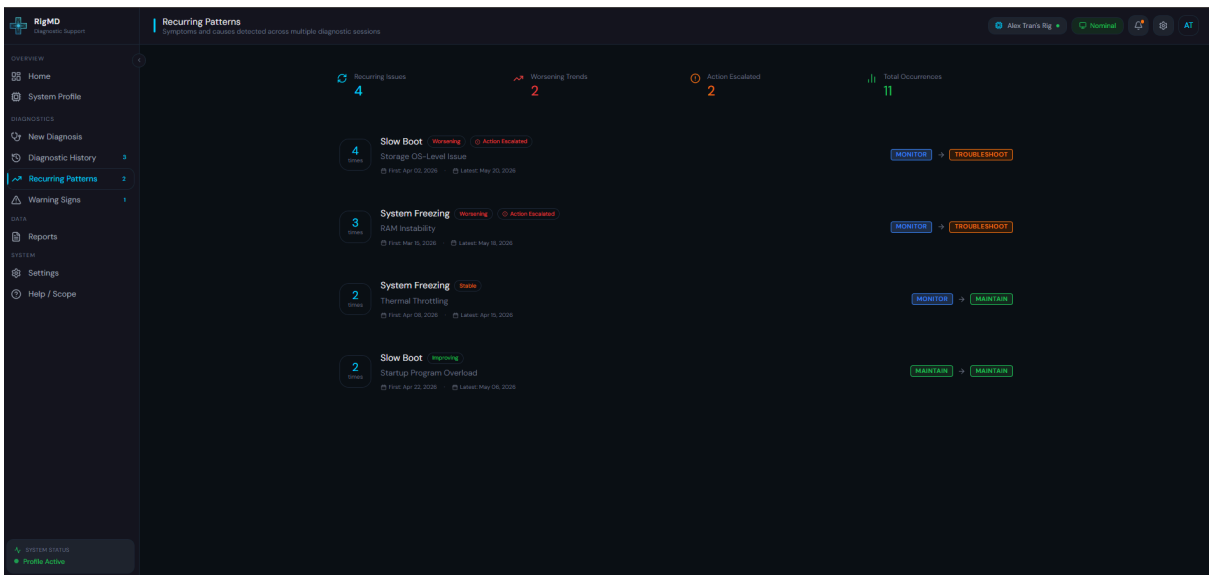


Table 3.6.2A. Front-end Components for Recurring Patterns

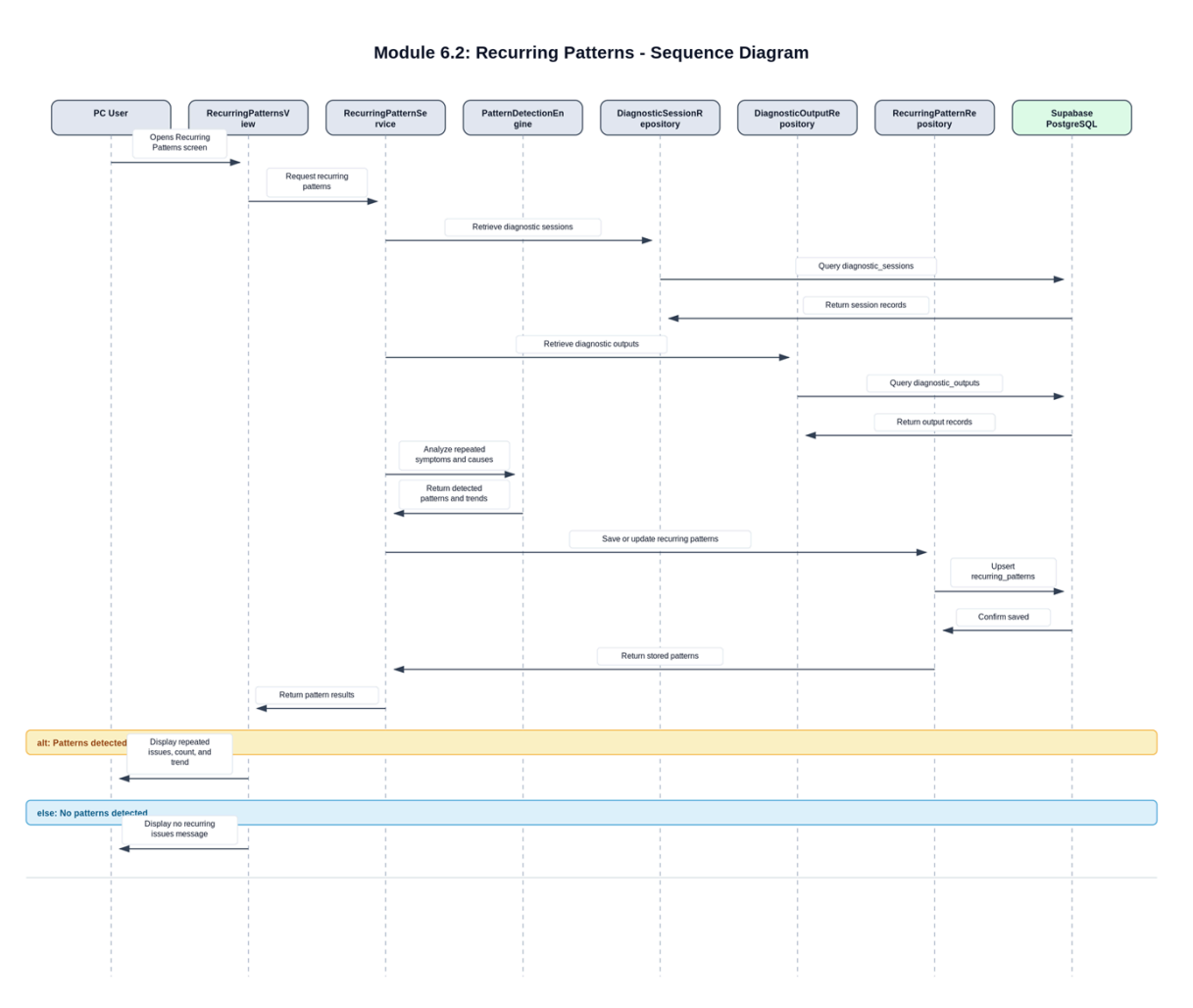
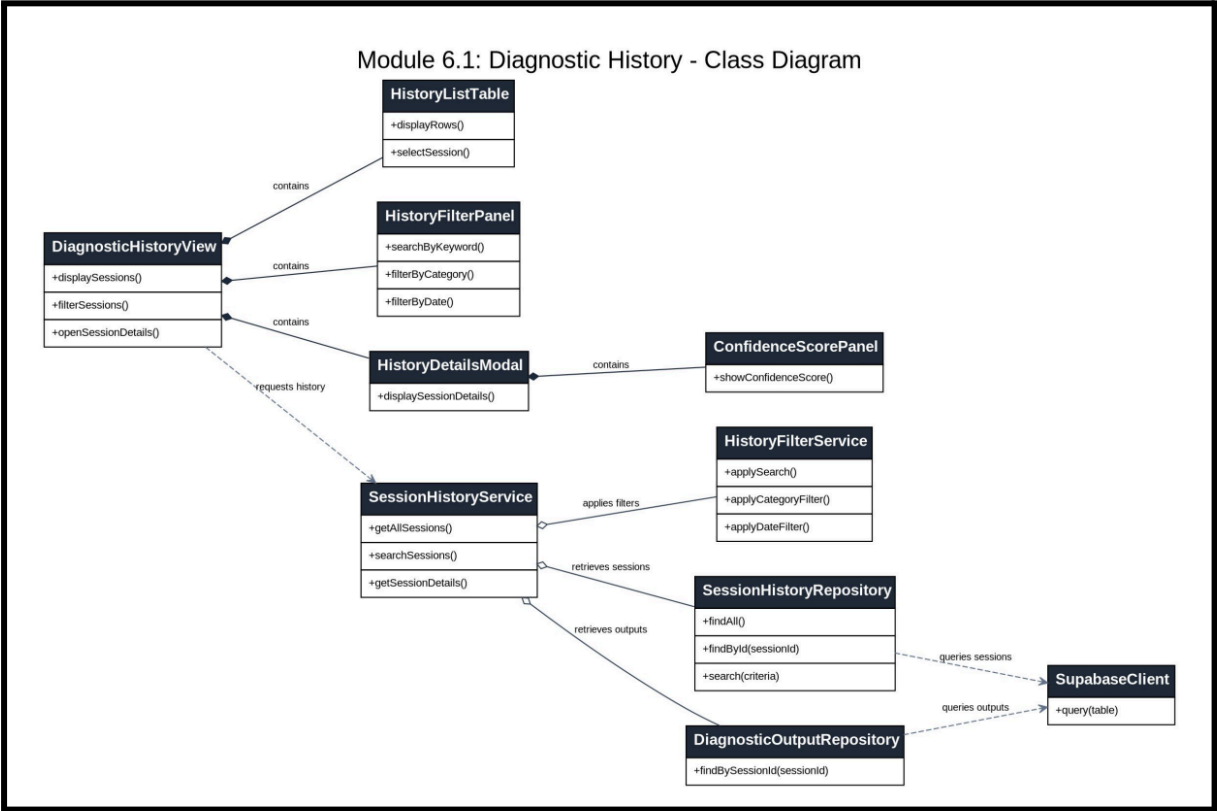
Component Name	Description and Purpose	Component Type / Format
RecurringPatternsView	Displays repeated symptoms, repeated cause categories, occurrence count, and trend labels.	CustomTkinter Screen / Frame
PatternSummaryCard	Shows the main repeated issue detected by the system.	CustomTkinter Card

PatternTrendCard	Displays whether the issue is stable, improving, or worsening.	CustomTkinter Card
PatternHistoryTable	Lists previous sessions connected to the recurring pattern.	CustomTkinter Table
NoPatternMessagePanel	Displays a message when no recurring pattern is detected.	CustomTkinter Panel

Table 3.6.2B. Back-end Components for Recurring Patterns

Component Name	Description and Purpose	Component Type / Format
RecurringPatternService	Detects recurring symptoms and recurring internal cause categories.	Python Service Class
PatternDetectionEngine	Compares diagnostic history records and calculates trends.	Python Engine Class
RecurringPatternRepository	Saves and retrieves recurring pattern records from Supabase PostgreSQL.	Supabase PostgreSQL Repository Class
DiagnosticSessionRepository	Provides session history data for pattern analysis.	Supabase PostgreSQL Repository Class
DiagnosticOutputRepository	Provides cause categories and action outcomes for trend comparison.	Supabase PostgreSQL Repository Class

Object-Oriented Components



Data Design / ERD or Schema for Recurring Patterns

